HAROKOPIO UNIVERSITY

SCHOOL OF DIGITAL TECHNOLOGY
DEPARTMENT OF INFORMATICS AND TELEMATICS
POSTGRADUATE PROGRAMME INFORMATICS AND TELEMATICS
COURSE WEB TECHNOLOGIES AND APPLICATIONS

Secure Location Sharing Platform for Closed Groups Using Home Assistant and Encrypted VPN Connection

Master Thesis

Anastasios Kotronis

Athens, 2025

ΧΑΡΟΚΟΠΕΙΟ ΠΑΝΕΠΙΣΤΗΜΙΟ

ΣΧΟΛΗ ΨΗΦΙΑΚΗΣ ΤΕΧΝΟΛΟΓΙΑΣ
ΤΜΗΜΑ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ ΤΗΛΕΜΑΤΙΚΗΣ
ΠΡΟΓΡΑΜΜΑ ΜΕΤΑΠΤΥΧΙΑΚΩΝ ΣΠΟΥΔΩΝ ΠΛΗΡΟΦΟΡΙΚΗ ΚΑΙ ΤΗΛΕΜΑΤΙΚΗ
ΚΑΤΕΥΘΥΝΣΗ ΤΕΧΝΟΛΟΓΙΕΣ ΚΑΙ ΕΦΑΡΜΟΓΕΣ ΙΣΤΟΥ

Ασφαλής Πλατφόρμα Διαμοιρασμού Γεωγραφικού Στίγματος για Κλειστές Ομάδες με Χρήση Home Assistant και Κρυπτογραφημένη Σύνδεση VPN

Μεταπτυχιακή Εργασία

Αναστάσιος Κοτρώνης

Αθήνα, 2025

HAROKOPIO UNIVERSITY

SCHOOL OF DIGITAL TECHNOLOGY
DEPARTMENT OF INFORMATICS AND TELEMATICS
POSTGRADUATE PROGRAMME INFORMATICS AND TELEMATICS
COURSE WEB TECHNOLOGIES AND APPLICATIONS

Examining Committee

Dimopoulos Alexandros (Supervisor)
Assistant Professor, Department of Informatics and Telematics, Harokopio University

Tsadimas Anargyros (Examiner)
Teaching Laboratory Staff, Department of Informatics and Telematics, Harokopio University

Dimitrakopoulos Georgios (Examiner)
Professor, Department of Informatics and Telematics, Harokopio University

Ethics and Copyright Statement

I, Anastasios Kotronis, hereby declare that:

1) I am the owner of the intellectual rights of this original work and to the best of my knowledge, my work does not insult persons, nor does it offend the intellectual rights of third parties.

2) I accept that the Library and Information Centre of Harokopio University may, without changing the content of my work, make it available in electronic form through its Digital Library, copy it in any medium and / or any format and hold more than one copy for maintenance and safety purposes.

3) I have obtained, where necessary, permission from the copyright owners to use any third-party copyright material reproduced in the master thesis while the corresponding material is visible in the submitted work.

Dedication

To my mother Kostoula, my uncle Thanasis, and my friend Akis who left us too soon.
To my partner Ioanna for her love and care.

Acknowledgements

I would like to sincerely thank my supervisor, Assistant Professor Alexandros Dimopoulos, for his kindness, guidance, and availability throughout the course of this thesis.

TABLE OF CONTENTS

Abstract in Greek

Η παρούσα εργασία παρουσιάζει τον σχεδιασμό και την υλοποίηση μιας self-hosted πλατφόρμας για τον ασφαλή διαμοιρασμό γεωγραφικού στίγματος εντός κλειστών ομάδων χρηστών, με χρήση εργαλείων ανοιχτού λογισμικού και εικονικής υποδομής. Κύριος στόχος είναι η ανάπτυξη ενός συστήματος που επιτρέπει στους χρήστες να μοιράζονται με ασφάλεια τη θέση τους σε πραγματικό χρόνο με μέλη της ομάδας, να ορίζουν γεωφράχτες και να ενεργοποιούν έξυπνες IoT ενέργειες βάσει γεωχωρικών συμβάντων.

Το σύστημα φιλοξενείται σε μια εικονική μηχανή **Linux**, στην οποία το εργαλείο **Docker Compose** ενορχηστρώνει διάφορες υπηρεσίες: μία backend υπηρεσία **Django** με **PostgreSQL/PostGIS** για αποθήκευση δεδομένων, έναν πίνακα ελέγχου **Streamlit** για παρακολούθηση, το **Home Assistant** για ενσωμάτωση IoT συσκευών, και έναν διακομιστή **VPN WireGuard** με περιβάλλον διαχείρισης. Για την ασφαλή απομακρυσμένη πρόσβαση, χρησιμοποιούνται τα reverse proxy εργαλεία **Localtonet** και **Ngrok**.

Η κινητή εφαρμογή έχει αναπτυχθεί με χρήση του εργαλείου **Kivy** και της βιβλιοθήκης **KivyMD**. Οι χρήστες δημιουργούν ένα ασφαλές VPN τούνελ μέσω της εφαρμογής WireGuard, χρησιμοποιώντας ρυθμίσεις που δημιουργούνται από το περιβάλλον διαχείρισης του WireGuard. Αφού συνδεθούν, η εφαρμογή επικοινωνεί με την υπηρεσία Django μέσω του VPN και στέλνει ενημερώσεις τοποθεσίας που αποθηκεύονται στη βάση PostgreSQL.

Μέσω του πίνακα ελέγχου Streamlit, οι χρήστες μπορούν να δημιουργούν ιδιωτικές ομάδες, να προσκαλούν μέλη και να ορίζουν γεωφράχτες. Η παρακολούθηση τοποθεσίας σε πραγματικό χρόνο επιτυγχάνεται μέσω περιοδικών αιτήσεων στα endpoints της backend υπηρεσίας. Όταν ένας χρήστης εισέρχεται ή εξέρχεται από μια καθορισμένη ζώνη, ενεργοποιούνται συμβάντα ειδοποίησης, τα οποία προωθούνται στο frontend και ταυτόχρονα στο Home Assistant. Μία έξυπνη πρίζα **TP-Link Tapo** έχει ενσωματωθεί για την επίδειξη αυτοματισμών που ενεργοποιούνται από γεωχωρικά συμβάντα, με ενεργοποίηση ή απενεργοποίηση της παροχής ρεύματος κατά την είσοδο ή έξοδο από ζώνη.

Λέξεις κλειδιά: Διαμοιρασμός γεωγραφικού στίγματος, Οικιακός αυτοματισμός, VPN

Abstract in English

This thesis presents the design and implementation of a self-hosted platform for secure geolocation sharing within closed user groups, using open-source tools and virtualized infrastructure. The primary objective is to develop a system that enables users to share their real-time location securely with group members, define geofenced zones, and trigger smart IoT actions based on geolocation events.

The system is deployed inside a **Linux** virtual machine where **Docker Compose** orchestrates several services: a **Django** backend with **PostgreSQL/PostGIS** for data storage, a **Streamlit** dashboard for monitoring, **Home Assistant** for IoT integration, and a **WireGuard VPN** server with a management interface. To ensure secure remote access, the setup employs **Localtonet** and **Ngrok** as reverse proxy tools.

The mobile component is developed using the **Kivy** framework and **KivyMD**. Users establish a VPN tunnel using the WireGuard mobile app using client configurations generated from the WireGuard UI. Once connected, the Kivy-based app communicates with the Django backend over the tunnel, sending location updates to be stored in PostgreSQL.

Through the Streamlit dashboard, users can form private groups, invite others, and define geofenced zones. Real-time user location monitoring is achieved through periodic polling of backend endpoints. When users enter or exit a defined zone, notification events are triggered and pushed to the frontend, and also forwarded to Home Assistant. A **TP-Link Tapo smart plug** is integrated to demonstrate geolocation-triggered automation by toggling its power state upon specific zone transitions.

Keywords: Location sharing, Home automation, VPN

LIST OF FIGURES

Abbreviations

VPN	Virtual Private Network
IoT	Internet of Things
ORM	Object Relational Mapping
DRF	Django Rest Framework
VM	Virtual Machine
UI	User Interface
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
UDP	User Datagram Protocol
API	Application Programming Interface
GIS	Geographic Information System
GPS	Global Positioning System

Chapter 1: Introduction

1.1: Background and Motivation

Connected! If a single word had to be chosen to describe the constant situation every unit of our modern society is in, this would be a mostly accurate one. From people to all kinds of devices, vehicles and infrastructures, from social networks, mobile communications, secure digital transactions and data transfer to location tracking, logistics, fleet management, social services and automations, in every context in general, the most prevalent concept is this of connectivity.

Of the various applications used in the aforementioned contexts, location tracking and smart devices constitute a growing trend in the last few years.
Delivery services give the ability to track items` location, health monitoring applications track user location to provide activity statistics, smart devices are used in home automations of all kinds to make the life of users easier and more comfortable.

Most such commercial applications, however, rely on centralized services with limited privacy guarantees and provide minimal flexibility and control in this regard.

As the title of this thesis suggests, we explore the design and implementation of a self-hosted, secure, extensible, configurable and modular location sharing platform.
The system supports closed user groups, enables zone-based monitoring (geofencing) and integrates with home automation tools to trigger actions based on location-based events.

To achieve this goal, a variety of modern open source tools are used that include microservices built with **Docker**, like **WireGuard** VPN, **Home Assistant** and a mobile application developed with the **Kivy** and **KivyMD** python frameworks. The motivation stems from the need for a privacy preserving platform that gives users control over their own data and infrastructure, offers an interactive user experience while providing an extensible solution loosely coupled with existing tools that may already support similar features but confine users to their own product policies and choices.

1.2: Objectives

A crucial first step on each software development process that paves the way through it and determines the architectural and technical decisions to be made, is a breakdown of the objectives that, themselves, follow from the desired scenarios our system should serve. So a succession of questions should arise naturally, the answer to any of which defines the next ones and their answers, leading to the objective breakdown and eventually the implementation:

- What is the basic feature we would like to provide a user using our system with?

  Answer: Monitor its location.

Objective 1: Ok, everyone has a mobile device these days, so lets build a basic mobile app that will share gps location.

- Do we want security over the location sharing process?

  Answer: Yes.

Objective 2: Using encrypted tunnels over a secure VPN connection could be a good idea for this regard.

- Could we avoid host imposed deployment configurations and potential fees?

  Answer: Examine self hosting possibilities.

Objective 3: Let’s go for building our system on our own computer using it as a server.

- How would data reach our self hosted system from outside of our home network?

  Answer: Router port forwarding could be one option, reverse proxies another one.

Objective 4: Bring into play a reverse proxy service which could avoid dependence on router configuration.

- Where would our data go?

  Answer: A backend service should be the way to go here.

Objective 5: Examine options out of various programming languages and frameworks according to personal preferences.

- Location monitoring should be visualized somehow right?

  Answer: Of course, if we want our system to have a practical use.

Objective 6: Building a front end dashboard for visualization and management is a must.

- Location-based events were mentioned. Should the user be notified of them?

Answer: Yes, if we want users to be aware of when events took place and why event triggered actions occurred.

Objective 7: We can explore the implementation of a notification feature.

- What about storage? What kind of data are we dealing with?

  Answer: We are talking about location sharing, thus geospatial data (*What Is Geospatial Data?*, n.d.), and geofencing support.

Objective 8: A database service supporting these concepts is required. PostgreSQL is such a database, widely used and reliable. Maybe we could use this.

- Should home automation and event based actions be a feature of our system?

  Answer: Yes.

Objective 9: Let's explore free tools along these lines. A popular and feature rich tool for such scenarios is Home Assistant. We can utilize this as well.

- What could be a minimal demonstrable end-to-end use case scenario of an event-triggered action?

  Answer: Maybe some kind of device could respond to a Home Assistant configured trigger.

Objective 10: Home Assistant is a tool providing this kind of functionality so we could use this to turn on and off the power supply of a simple device like a Tapo smart plug based on a configured trigger which might be fired by an external event like entering or exiting predefined zones.

1.3: Scope, Tools and Limitations

Now that with our objectives definition, what we are about to build starts getting clearer, it is important to point out that our purpose is by no means to present the process of building a production-ready system, but rather to focus on the development of a proof-of-concept solution, demonstrating the technical feasibility, available tool options, and the integration and interaction of different independent components.

The system requirements and the design choices in the production grade setting may differ due to concerns like scalability, stability, resiliency and availability among others. In contrast, our proof-of-concept case focuses on quicker results that are still decently presentable, using a

range of tools that can be handled by a single developer without requiring deep expertise in each single one.

With the above in mind, and considering the thesis objectives, an outline of the tools chosen for the individual application components is given below, each one of which will be discussed in dedicated sections:

- Development will be done in a linux virtual machine set up with **Vagrant** and **VirtualBox**.
- **Kivy** and **KivyMD** python frameworks will be used for the mobile application implementation.
- With **WireGuard** mobile application we will create tunnels establishing secure connections between the mobile application and the self hosted services.
- **Docker** and **Docker Compose** will be used for building and running components as microservices. These will include:
    - A python **Django** application as backend.
    - A **PostgreSQL** database service for data storage.
    - A python **Streamlit** application as frontend.
    - A **WireGuard** VPN service with its user interface to securely accept incoming traffic from outside the home network.
    - Two **reverse proxy** services acting as bridges between the self hosted WireGuard service and its user interface and the outside world. Specifically we use **localtonet** and **ngrok** respectively.
    - **Home Assistant** service to perform event triggering actions on connected devices.

Proof-of-concept systems have by definition inevitable limitations, and, as such, this is the case for our system as well. Some limitations in terms of architecture and implementation can be summarized as follows:

- Streamlit is a framework that offers rapid prototyping and the ability to produce decent visual results without having to deal with styling tools like CSS or Javascript. However it is mostly not recommended for production as it offers limited support for state management, follows a linear UI flow (re-executing the entire script on every UI interaction), and does not scale well under high load.

- Location monitoring from the dashboard built with Streamlit will involve real time notification functionality which will be implemented with **polling** technique. This is not a bad choice in the context of a proof-of-concept but in production grade solutions other techniques should be preferred instead, like **WebSockets** or **server-sent events** (**SSE**).
- Although Kivy and KivyMD frameworks support application development for iOS devices, our case is built for Android and tested only on this.
- Scalability concerns and testing the system under high loads of input is outside of the scope of this thesis.
- The use of third party tunneling services like localtonet or ngrok serve just fine the purpose of our application but are not ideal for long term deployment or production as they depend on external infrastructure and may have usage limitations or reliability issues.
- VPN configuration can be considered semi-manual as it requires the user to download WireGuard mobile application and load a file to create a secure tunnel connection in order to access the self hosted services.

1.4: Document Structure

- **Chapter 1: Introduction** outlines the motivation behind the project, the main objectives, the scope and limitations of the work, the tools used, and an overview of the document structure.
- **Chapter 2: Theoretical Background and Related Concepts** provides a technology review that contextualizes the project. It explores core concepts such as location-based services, geofencing, VPN technologies (with a focus on WireGuard), smart home automation, and the role of containerization through Docker. It also examines existing platforms and highlights how this work is differentiated from or builds upon them.
- **Chapter 3: System Requirements and Design** details the functional and non-functional requirements of the system, outlines the roles of different users, and presents use cases that guide the system's behavior and capabilities.
- **Chapter 4: System Architecture and Interaction** presents an in-depth description of the system's architecture, its components, and how they interact. It includes networking details, data flow between services, the VPN setup, and the overall environment used for deployment. Each subsystem is explained in terms of its function, integration, and role in the complete stack, from the mobile application and backend to the Streamlit

dashboard, Home Assistant, and the use of third-party tunneling services such as Localtonet and Ngrok.

- **Chapter 5: Implementation** outlines some of the core implementation details, focusing on how the system was put together in practice. It links architectural decisions with the actual tools, frameworks, and code used.
- **Chapter 6: Conclusion and Future Work** presents an overview of the implemented system, highlighting its architecture, functionality, and integration aspects. It then discusses the challenges faced during development, outlines directions for future improvement, and concludes with closing remarks reflecting on the contributions of the thesis.

Chapter 2: Theoretical Background and Related Concepts

2.1: Location-Based Services and Geofencing

In the era of connectivity and personalization, every entity, whether living or nonliving, is being closely monitored and the range of information collected and analysed is constantly expanding. Location, of course, is no exception.

While the question of who benefits from this might be a subject of debate, depending on the context in which this is done, Location-Based technology uses geographical information to provide services to users, so a *Location-Based service* is a software application that utilizes this technology for its services (*Location-Based Service*, n.d.).

The range of use is vast and it varies from marketing, e-commerce and personalized advertising, to entertainment and social networking, from logistics and navigation to naval industry, from personal security to health services, from the arms and defence industry to military purposes in general and much more.

The term *geofencing* in relevance to the above context refers to the dynamically created or predefined virtual geographical border, or "fence" around a physical or conceptual entity that may also be called geographic feature (*Geofence*, n.d.) (*Geographical Feature*, n.d.).

Location data can be obtained in various ways, the most common of which, that we will also implement in this thesis, is via mobile devices. In these cases, the device typically communicates its location through navigation satellites or mobile radio towers and routers (*What Is a Geofence? How Does Geofencing Work?*, n.d.).

Specifically for the geofencing part in our implementation, the user will use the dashboard to define predetermined zones on a map as polygons and specific actions will be triggered when the user enters or exits these zones.

2.2: Security and Privacy in Location-Based Services

Location data is highly sensitive information, especially when living entities are involved, so sharing it raises serious concerns regarding privacy, data interception and unauthorized monitoring. A secure location-based service architecture should be designed in a way that ensures:

- End-to-end encryption between communicating parties,
- Authorized access control to exchanged data,
- Data anonymization to the greatest possible extend,
- Minimal reliance on third party services where user control is limited.

The implementation of this thesis reflects the above concerns by enforcing VPN-based encrypted communication with WireGuard, providing location sharing among closed groups, and avoiding cloud based services through self-hosting.

2.3: VPN Technologies and WireGuard

Virtual Private Network, or VPN, is a communication mechanism that extends a private network, such as a home or corporate network, across other networks. By establishing an encrypted connection it provides secure access to the private networks for entities that do not have direct access.

In the most common use cases, a VPN is used to provide remote access to protected resources, such as enabling employees to work from home, while ensuring confidentiality and protecting data from interception during sensitive data transfers. It can also be used to bypass regional restrictions, prevent tracking by internet service providers or third parties and ensure privacy by masking IP addresses (*What Is a VPN? Why Should I Use a VPN?*, n.d.) (*Virtual Private Network*, n.d.).

There are various options when it comes to selecting a VPN tool and the choice depends on the needs of each use case. Some of the aspects one might consider include whether the tool is free or paid, whether it supports self hosting, ease of setup and configuration, the amount of free data offered, its speed and flexibility, as well as whether it is open-source, which allows for greater transparency and control, or proprietary, which may come with restrictions or licensing costs.

Some popular free choices that support self hosting apart from WireGuard include **OpenVPN**, **StrongSwan** and **SoftEther** (*OpenVPN*, n.d.) (*StrongSwan*, n.d.) (*SoftEther*, n.d.).

For the purpose of this thesis WireGuard was selected not only because it is free and supports self hosting, but also because the Docker image we used provides an intuitive and easily configurable user interface through which various management operations can be performed such as:

- Global configuration and traffic routing,
- User Management,
- Client definition,
- QR Code generation for client configurations
- Client configuration exports as `.conf` files

(*Linuxserver/wireguard*, n.d.)

Additionally, the Docker service integrates excellently with WireGuard mobile application allowing users to create connections easily either by using the exported configuration files or by scanning the generated client QR Codes.

2.4: Smart Home Automation and IoT

The word *automation*, which has Greek origins, conveys the concept of self-reliability. In the context of technology it refers to a process that does not require human intervention. It typically aims at reducing costs, saving working hours, improving quality of services and products, enhancing security and control, among other benefits that depend on the area of application (Frey & Osborne, n.d.).

Home automation, as the name suggests is the application of the self-reliability concept in the context of home. This might include the configuration and management of aspects like energy or water consumption, the control of electricity supply, security setup and others (*Home Automation*, n.d.).

Specifically the term *smart home automation* refers to the application of home automation using devices that can be controlled wirelessly, or, more specifically, that have internet access and can communicate over the internet (*Home Automation*, n.d.).

The device components used in smart home automation applications constitute what we refer to with the term *Internet of Things*, or *IoT*. These devices typically include embedded software, computing and processing capabilities and the ability to communicate with other systems and exchange information.

Such devices are usually managed and controlled through centralized services which could be cloud-based or self-hosted. In this thesis we will use the Home Assistant platform for this purpose and a Tapo smart plug as a minimal example of how power supply can be triggered on and off by platform configured triggers fired by external events.

2.5: Related Projects and Platforms

There are many existing tools and platforms that support location sharing and smart home automation features. One of the most popular is Home Assistant with device auto discovery feature, device location monitoring, geofencing by zone definition and event driven automation such as entering or exiting zones and even WireGuard integration as an addon.

Other platforms with similar functionality include **OpenHAB** (*OpenHAB*, n.d.) which is also popular, free, open source and supports self-hosting as well as a vast range of devices and **Google Home** (*Google Home*, n.d.) a cloud-based alternative that is part of the Google ecosystem, not self-hosted and offers very limited user control. **Node-RED** (*Node-RED*, n.d.) is also worth mentioning since, even though it is not a complete smart home automation platform, it is widely used to automate smart devices and orchestrate workflows through a user-friendly node-based user interface.

Despite the fact that existing platforms already support similar features, the purpose of this thesis is to independently implement some of these capabilities and demonstrate, from both a planning and development standpoint, how various tools can be used and individual components can be integrated. The goal is to create an application with standalone, reusable and extensible components providing finer control during the development process as well as to the end user.

2.6: Monoliths, Microservices and Docker

With the evolution of software development throughout the years, the way systems are structured continuously adapted, not only in response to technological advancements and shifting market demands, but also due to increased complexity, scalability requirements and the growing emphasis on flexibility, deployment speed and maintainability. These changing conditions gave rise to new architectural paradigms and tools. Some of the stages of this evolution are reflected in the following concepts.

2.6.1: Monolithic Architecture

Older and traditional systems followed the paradigm of standalone, self-contained units independent of separate components, typically with a single code base, where different parts, such as backend logic, frontend interface and database configuration were bundled in a single entity and operated together as a unified whole. This kind of architecture is referred to as *monolithic*.

Although this paradigm offers some advantages, such as easier debugging, development and deployment process, it can slow development, due to a larger codebase, reduce flexibility because of its tightly coupled components and make scaling difficult (Harris, n.d.).

2.6.2: Microservices Architecture

The rapid growth of digital transformation in the first years of the new millennium shifted this traditional paradigm to a new one where the primary concerns were flexibility, scalability and reliability. Systems built with this model consist of separate, independently deployable services where each one can be developed and managed without affecting the others making working teams autonomous, complexity manageable, scalability, resilience and reliability more feasible. One of the first companies to follow this architecture migration was Netflix in 2009 (Harris, n.d.).

Despite the problems it solves, a microservices architecture introduces its own challenges such as increased infrastructure costs, inter-service communication complexity and operational overhead.

2.6.3: Containerization, Docker and Docker Compose

The term *containerization* refers to a deployment method in which an application's code and dependencies, along with necessary operating system libraries, are bundled together so that the application can run consistently in any infrastructure in an isolated environment called *container* (*What Is Containerization?*, n.d.) (*What Is Containerization? - Containerization Explained*, n.d.).

Docker is a widely used containerization tool that plays a key role in implementing a microservices architecture, as it allows developers to build, ship and run containers. Applications are packaged into Docker *images* which can be stored and distributed through Docker *image registries* such as Docker Hub. Docker also supports persistent storage through *volumes,* which are independent of a container's lifecycle.

For the management of multi-container applications Docker provides *Docker Compose*. With this tool, all services can be defined in a *yaml* configuration file, each one with its own dependencies, environment variables, volumes, networks and other configurations so that they can be built and run with a single command.

Chapter 3: System Requirements and Design

3.1: Functional Requirements

The definition of the core functionalities our system provides is described in the following:

- **User Registration and Authentication**: Users can create an account in the system dashboard and login with their credentials to access individual pages.
- **Group Definition and Management**: Logged in users can:
    - Create a group,
    - Invite other users to this group,
    - Respond to group invitations by accepting or declining pending requests,
    - View the history of sent and received group invitations,
    - Delete a group in which they are a member.
- **Zone Definition and Management**: Logged in users can:
    - Create polygon-based zones on a map and associate them with the appropriate groups,
    - Delete zones associated with their groups.
- **Location data collection and monitoring**: GPS data are collected using a custom-developed mobile application built specifically for this system, and sent to the backend service through VPN connection. User locations are then displayed as markers on an interactive map within the dashboard.
- **VPN Management**: Users establish a secure VPN connection with the backend service using WireGuard mobile app. The tunnel can be created by either uploading a configuration file provided by the hosted WireGuard service or by scanning a generated QR Code.
- **Entry/Exit Detection and Notifications:** The backend detects when a user enters or exits a zone and sends real-time notifications to the frontend.
- **Smart Home Integration**: Home Assistant triggers predefined actions. Specifically, the power supply is turned on/off on a TP-Link smart plug based on the zone events.

(*Functional Requirement*, n.d.)

3.2: Non-Functional Requirements

Requirements that define the quality attributes and operational aspects of our system can be outlined in the below:

- **Security**: Communication security is established by VPN end-to-end encryption.
- **Privacy**: Access to location data is authorized on group level.
- **Usability**: KivyMD and Streamlit frameworks provide a clean and user friendly interface for both the mobile application and the dashboard.
- **Maintainability**: The Dockerized services establish modular architecture and allow easy maintenance, testing, debugging and extensibility.
- **Scalability**: While the system is not designed with scalability as the main concern, adjusting Docker services and infrastructure can help in this regard.

(*Non-Functional Requirement*, n.d.)

3.3: User Roles and Actions

The roles of users interacting with the system are not defined by strict authorization rules or access restrictions within the application itself, but rather emerge from the practical use-case scenarios. For example, a user acting as an administrator may manage VPN client configurations through the WireGuard interface and distribute them to users whose locations will be monitored. These users can then import the configurations into the WireGuard mobile app to establish a secure tunnel.

The WireGuard interface supports client configuration retrieval via QR Code, configuration file, email and Telegram (*Ngoduykhanh/wireguard-Ui: Wireguard Web Interface*, n.d.). In our implementation, the WireGuard UI is accessible on the user's mobile device through an ngrok reverse proxy. This allows end users to log in with their credentials and directly create client entries and download their configurations, provided that an administrator has granted them access to the WireGuard interface.

3.4: Use Case Description

The following use-case scenarios illustrate complete end-to-end flows supported by the system:

- **Use Case 1**: *WireGuard VPN Configuration*

  Actors: Administrator User, WireGuard Service UI

Description: The administrator either creates user accounts for individuals whose locations will be monitored, allowing them to log in to the WireGuard UI and generate/download their own client configurations, or directly creates the clients and distributes the corresponding configuration files via email.

- **Use Case 2**: *Group Creation and Management*

  Actors: Authenticated User, Streamlit Dashboard

  Description: An authenticated user creates a group, invites other users or responds to group invitations.

- **Use Case 3**: *Geofencing Zone Definition*

  Actors: Authenticated User, Streamlit Dashboard

  Description: An authenticated user draws polygon zones on a map for specific groups and submits them to the backend for zone monitoring.

- **Use Case 4**: *WireGuard VPN on the mobile side*

  Actors: User, Mobile Application

  Description: A user downloads the WireGuard mobile application from Google Play to establish secure connections with the backend using the client configurations.

- **Use Case 5**: *Location Data Sharing*

  Actors: Authenticated User, Mobile Application

  Description: After establishing a connection using the WireGuard mobile application, the user can authenticate via the mobile app and enable or disable the location sharing option to either send or stop sending GPS location data to the backend.

- **Use Case 6**: *Location Data Monitoring*

  Actors: Authenticated User, Streamlit Dashboard

  Description: An authenticated user selects groups from the dashboard and monitors the corresponding user locations on the map. When users enter or exit predefined zones, notifications are displayed on the dashboard.

- **Use Case 7**: *Trigger Smart Plug*

  Actors: Backend Service, Home Assistant

  Description: When the backend detects a zone entry/exit event, it triggers a webhook that toggles the Tapo plug's power supply via Home Assistant.

Below we can see a UML diagram of the aforementioned use cases created with the **draw.io** online tool (*Draw.io*, n.d.):

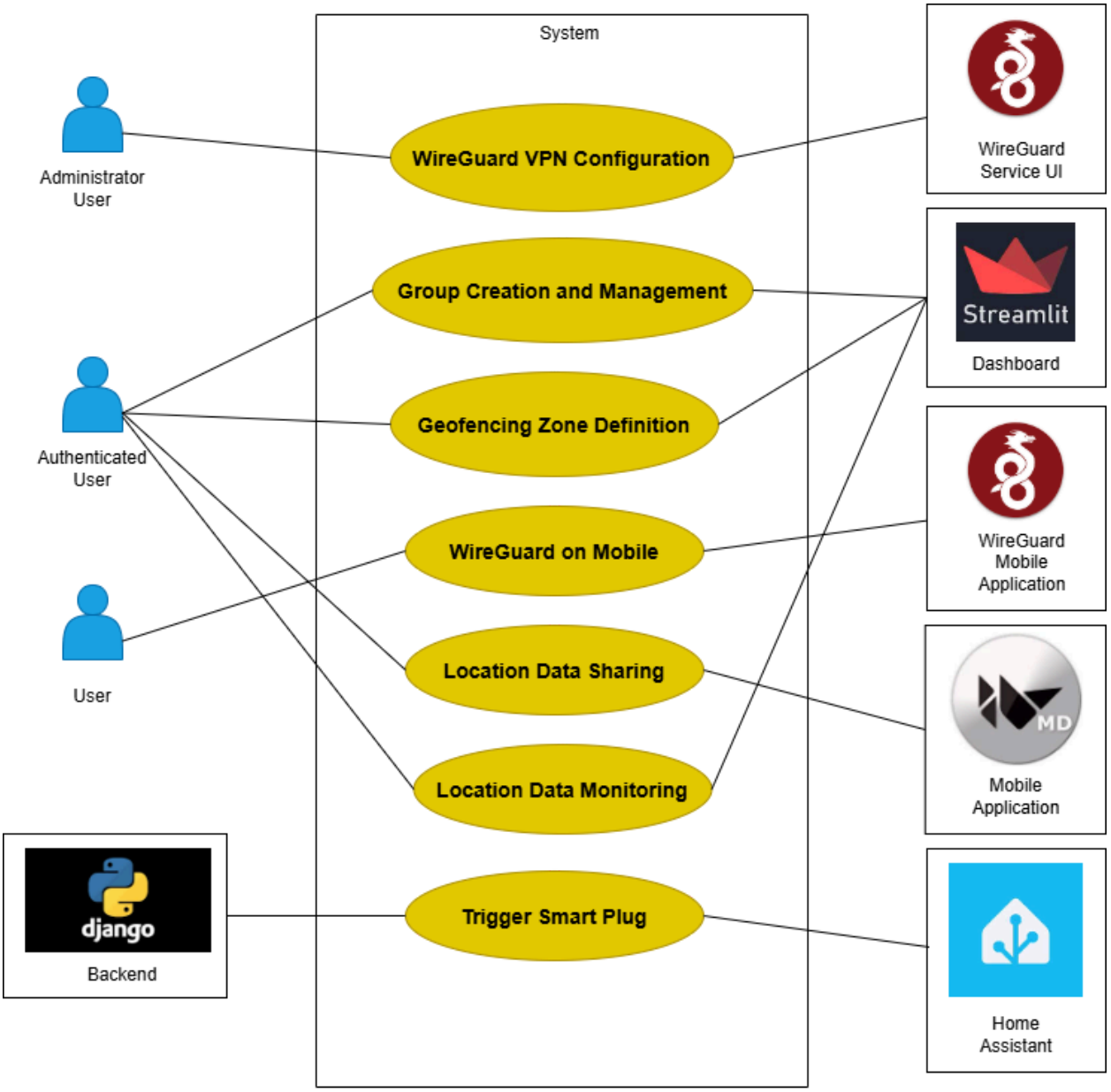

Figure 1: UML use case diagram

Chapter 4: System Architecture and Interaction

4.1: Overall Architecture Overview

As the outlined requirements suggest, our system architecture prioritizes privacy, security and simplicity over concerns typically associated with production-grade solutions, such as scalability and reliability.

While Django and PostgreSQL provide a solid foundation for building scalable and reliable backend systems, Streamlit is better suited for small to medium-scale deployments, academic prototypes, or self-hosted environments rather than large-scale production systems.

Although the Kivy framework, used for the mobile client, offers a decent development speed, it lacks the native look and feel found in commercial mobile frameworks. However, when combined with the KivyMD, which provides support for material design widgets, it offers a practical support to this limitation.

Wireguard is a high-performance and secure VPN solution. In the current setup we use it in conjunction with proxies to provide a connection method that avoids manual router configuration. This approach introduces some dependency on third-party services but simplifies deployment and remote access.

Finally, Home Assistant with its flexible ecosystem enables seamless integration with local IoT devices and automations such as the Tapo smart plug and its associated triggers.

The architecture depicted in the following diagram supports an approach that delivers meaningful functionality particularly in scenarios where privacy, autonomy, and ease of self-hosting are essential.

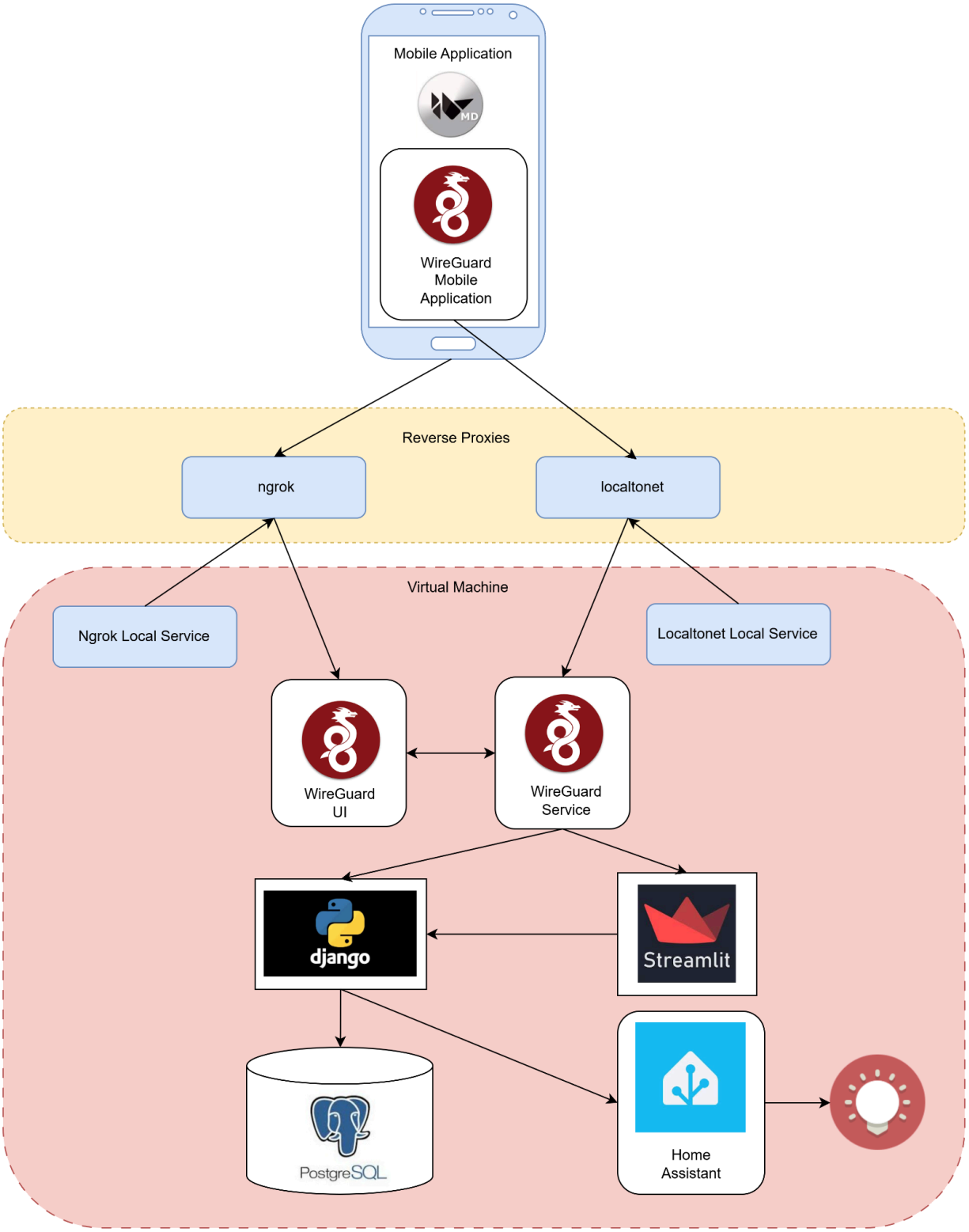

Figure 2: Overall architecture

4.2: Networking and Data Flow

The networking architecture of the system ensures secure, private, and seamless communication between the mobile client and the services hosted inside the Virtual Machine

(VM). The core mechanism enabling this communication is the use of a WireGuard VPN, which is exposed externally via reverse proxy tools.

In the following sections, we detail the steps involved in the end-to-end data flow.

4.2.1: Mobile-to-Backend Communication via VPN

In order for the mobile application to communicate with the self hosted backend service, users are required to download the WireGuard mobile application and use their client configurations to establish a VPN connection to the WireGuard service running inside the VM. This encrypted connection ensures that all data exchanged between the mobile device and backend systems remains secure.

Once the VPN is active, the mobile app can communicate directly with the services on the VM via internal IP addresses assigned by the WireGuard network.

4.2.2: VPN exposure through Reverse Proxy

Since the WireGuard server is hosted inside a VM that is not directly exposed to the internet and router port-forwarding is not used, a reverse proxy is required to allow mobile clients to reach the WireGuard endpoint externally.

This is achieved through localtonet, which runs as a local service inside the VM, is synced with localtonet cloud service and forwards incoming traffic from the internet to the WireGuard service's listening port. The mobile device connects to the VPN using a public tunnel URL provided by localtonet like

`udp://<random-string>.localto.net:<random-port>.`

Once the device is connected to the VPN, incoming traffic reaching the WireGuard server is routed to the network where the backend service is running via predefined routing rules on the WireGuard configuration.

4.2.3: Ngrok for WireGuard UI Access

The WireGuard UI, through which client configurations can be managed, is also hosted inside the VM. To make this interface accessible from outside for administrative purposes, Ngrok is used to expose its HTTP port securely.

With a similar syncing mechanism to localtonet and a fixed URL like `https://<random-string>.ngrok-free.app`, Ngrok provides a secure HTTPS tunnel to the UI which can be accessed from a browser without requiring changes to firewall or router settings.

This way a user can log in the WireGuard interface with his credentials and create their client configurations to download and use to connect with the backend service.

4.2.4: VPN Tunnel Creation

To simplify VPN connections from mobile devices, the WireGuard UI generates client configuration files that include public keys, allowed IP ranges, DNS settings, and the remote endpoint which, in our case, is the one provided by localtonet pointing to the WireGuard service.

A sample of such a file is shown below, where the term peer is used by WireGuard for what we refer to as *client*:

```
[Interface]
Address = 10.x.x.x/32
PrivateKey = <Private-Key-Value>
DNS = 1.1.1.1
MTU = 1450

[Peer]
PublicKey = <Public-Key-Value>
PresharedKey = <Preshared-Key-Value>
AllowedIPs = 0.0.0.0/0
Endpoint = <localtonet-server-domain>:<localtonet-server-port>
PersistentKeepalive = 15
```

Figure 3: WireGuard client configuration file

These configurations can be encoded as QR codes, which can then be scanned by the WireGuard mobile app for instant setup. This avoids manual file transfers and provides a user-friendly and error-resistant way to connect devices to the private network, even in environments with dynamic IP addresses or non-technical users. Alternatively, the configuration files can be imported directly to establish the tunnel connection.

For our workflow we will use the second approach.

4.3: System Components

In the current section, we explore in detail each of the system's individual components, describing their purpose, how they are integrated within the overall architecture, and their role in enabling secure and reliable functionality. Each subsection focuses on a specific part of the system, from the mobile application and backend services to the VPN setup, frontend interface, reverse proxy mechanisms, Home Assistant integration, and the development environment hosted inside a Virtual Machine.

4.3.1: Reverse Proxies

As outlined in our objectives, one of the primary goals is to make our self-hosted services accessible from outside the home network. A common approach to achieve this is by setting up port forwarding on the local router. However, this method is tightly coupled to a specific network configuration, meaning that if we want to deploy the system in a different location, we would need to repeat the setup, assuming we even have the necessary permissions.

This limitation can be addressed through the use of reverse proxy services. In this thesis, we utilize two such services: **Localtonet** and **Ngrok**. According to Localtonet's documentation, these kinds of services are especially useful in environments behind Carrier-Grade NAT (*Carrier-Grade NAT*, n.d.) or similar constraints, where port forwarding is not feasible (*Localonet - Docs*, n.d.).

We use the free tiers of Localtonet and Ngrok to accomplish the following:

- **Route UDP traffic** from the WireGuard mobile application through a URL provided by Localtonet, which maps to our self-hosted WireGuard VPN server,
- **Expose the WireGuard UI** to the internet via a URL provided by Ngrok, allowing users to authenticate, create, and download client configuration files.

The following figures present the dashboards of the respective platforms used to configure the features described above:

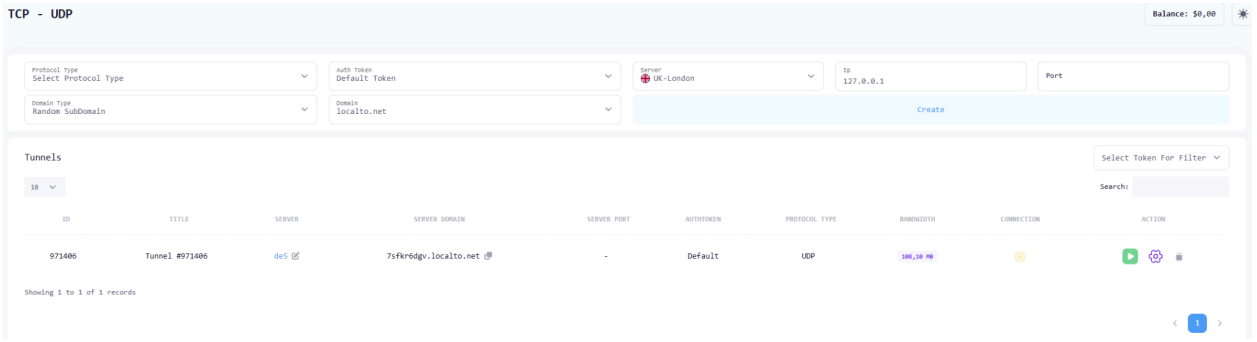

Figure 4: Localtonet dashboard

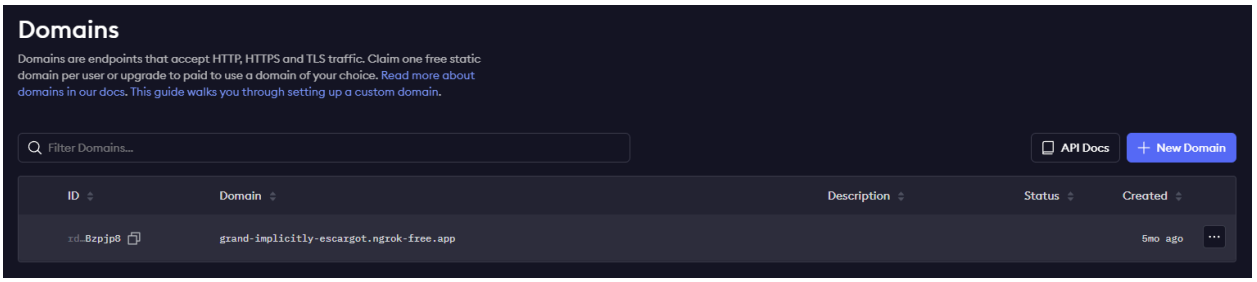

Figure 5: Ngrok dashboard

4.3.2: WireGuard VPN

With the reverse proxies configured, we established public accessibility to both the WireGuard VPN service and its corresponding UI. Although the UI can be accessed locally and client configurations can be set up by an administrator, we chose to expose it publicly in a secure manner for greater flexibility. Below is the UI as accessed through the public URL provided by Ngrok from the mobile device.

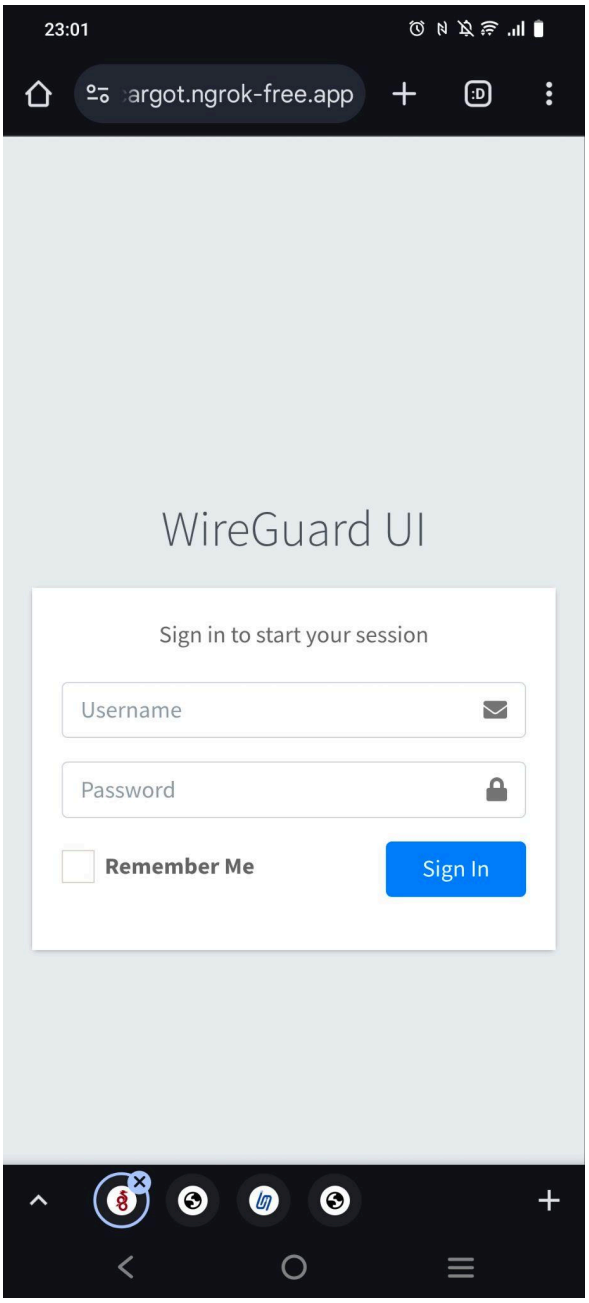

Figure 6: WireGuard UI accessed through the ngrok url from mobile device

After logging in, the user can configure the WireGuard VPN Global and Server Settings, if they are an administrator, as shown below:

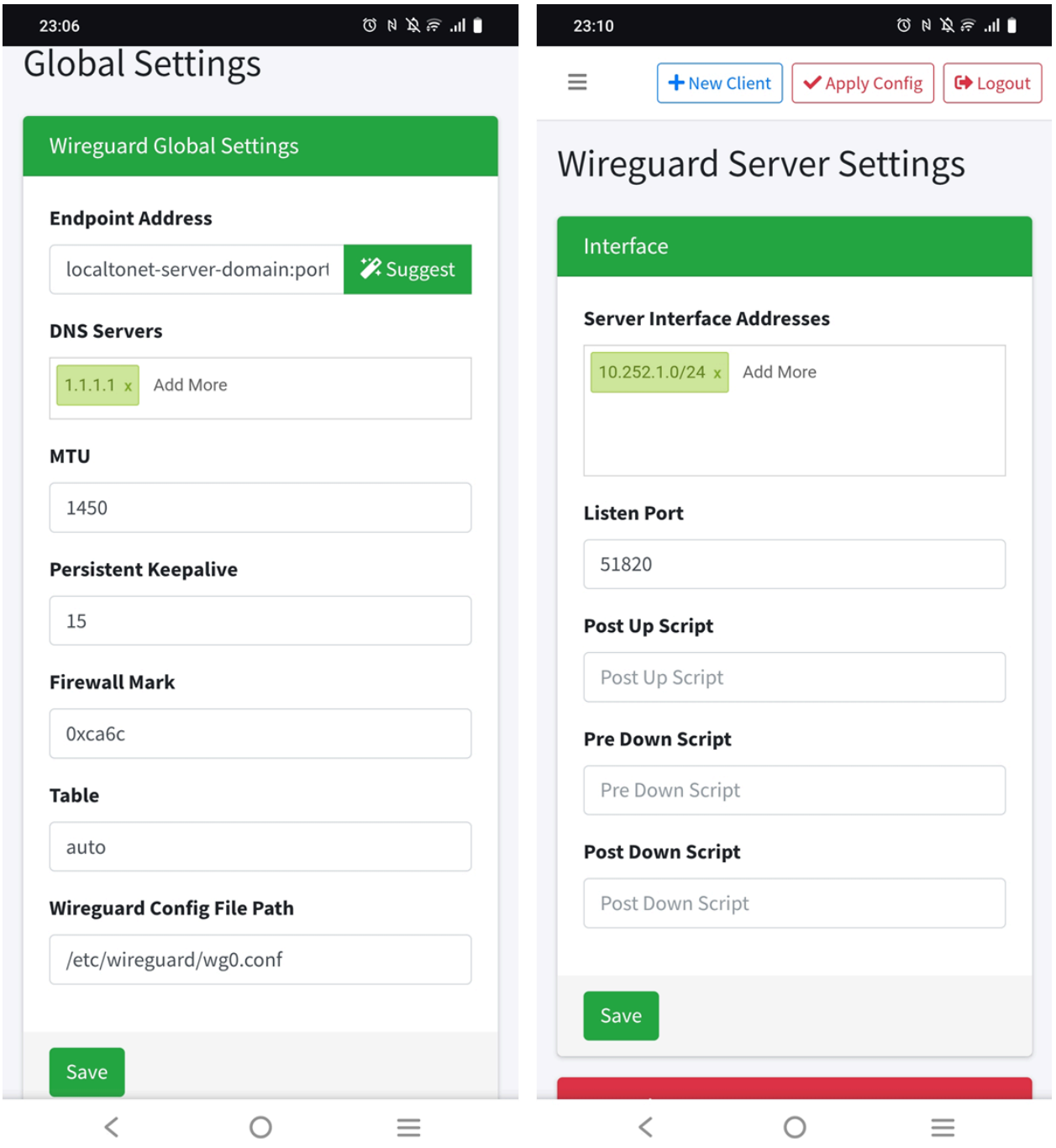

Figure 7: Global and server settings configuration through WireGuard UI

They can then create new clients and download the corresponding configuration files to use with the WireGuard mobile application:

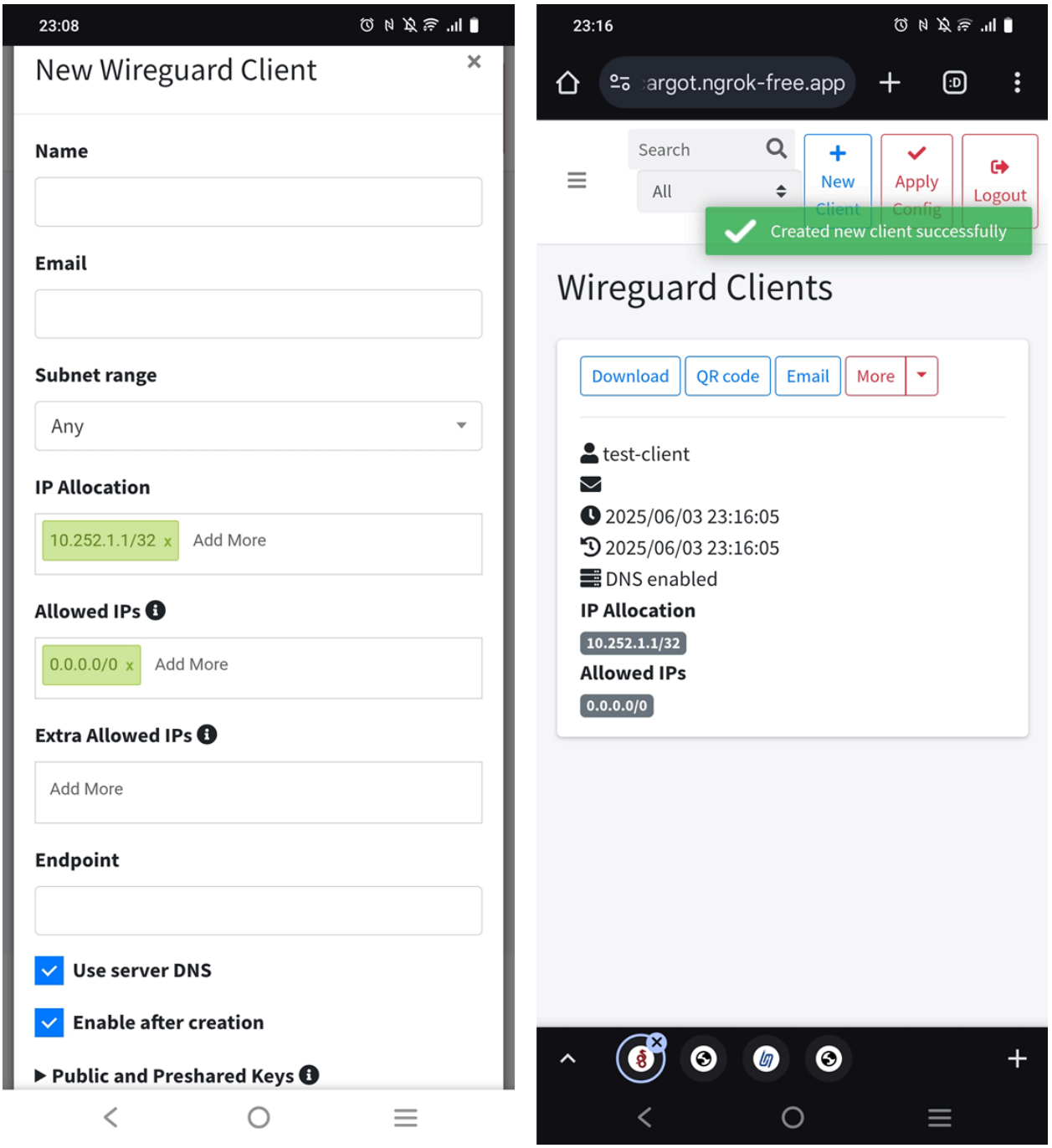

Figure 8: Client management through WireGuard UI

4.3.3: Backend Services

4.3.3.1: PostgreSQL and PostGIS

The microservice in our system that is responsible for data storage is a Dockerized PostgreSQL service. **PostgreSQL** is a powerful, open-source relational database system that has been in active development for more than 35 years. It has a widespread reputation in the developer community and, as of 2024, is the most popular database technology according to the *Stack*

Overflow Developer Survey (*Technology | 2024 Stack Overflow Developer Survey*, n.d.). Renowned for its rich feature set and extensibility, PostgreSQL emphasizes data integrity, fault tolerance, performance, and scalability (*PostgreSQL - About*, n.d.).

Two widely used PostgreSQL extensions, particularly relevant in the context of IoT applications, are **TimescaleDB** (for managing time-series data) and **PostGIS** (for geospatial data management) (*Top 8 PostgreSQL Extensions*, n.d.). TimescaleDB is optimized for managing time-series data and improves performance through automatic partitioning of data into **chunked hypertables** based on timestamps. PostGIS, on the other hand, enables robust geospatial data handling and includes functions for spatial queries, such as determining whether a specific GPS location lies within the interior of a polygon, which is particularly useful in our geofencing use case.

The seamless integration of PostgreSQL and its extensions with Django's ORM was a key factor in our choice of these backend technologies.

The following schema diagram, created with the **DrawSQL** online tool (*DrawSQL*, n.d.), presents the database schema. It outlines the core entities and their relationships as implemented in the system. Notably, the `Messages` table models group-level chats, but it is not included in the current code implementation. This component is intended as a potential extension for future work.

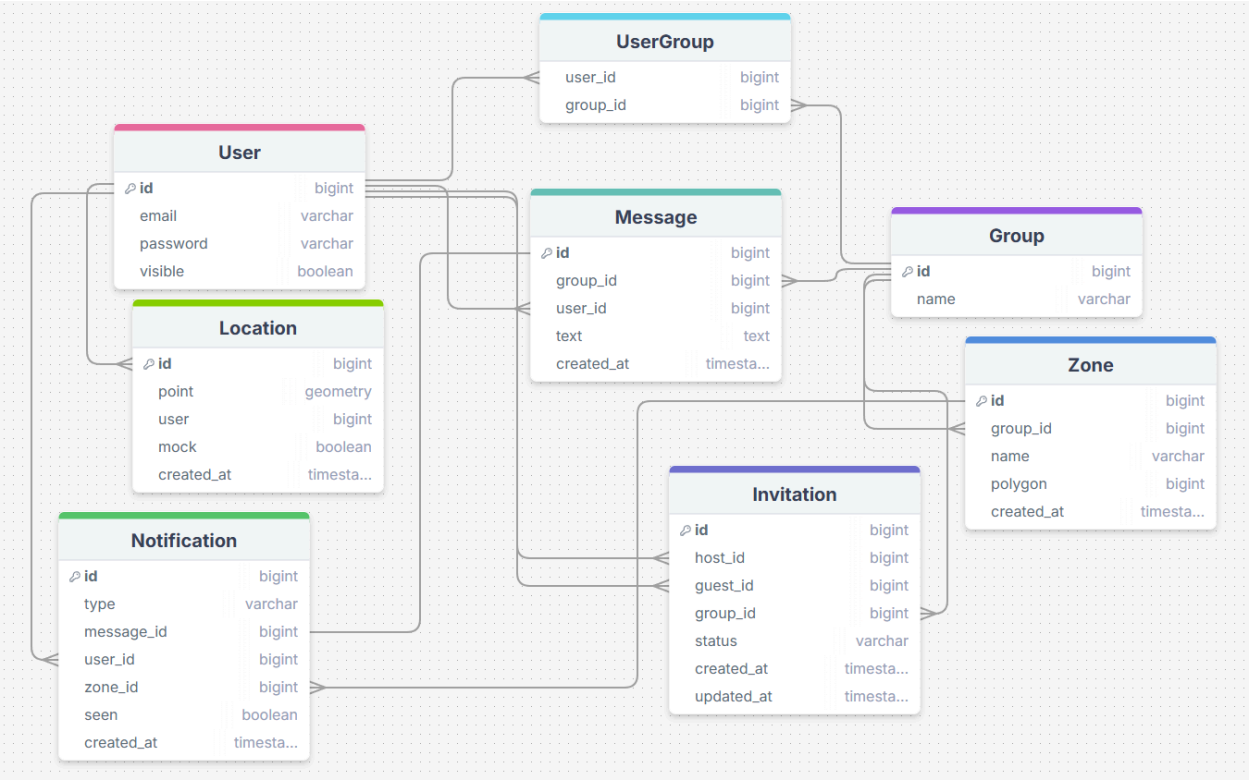

Figure 9: Database entities and relationships

4.3.3.2: Django and Django Rest Framework (DRF)

The backend service of our system is built using Django, a mature and reliable "batteries-included" framework for web development. Since its initial release in 2010, Django has been actively maintained and supported by a large and engaged community of developers. With 83.8k stars on GitHub at the time of writing—only recently surpassed by FastAPI (*Django RepositoryStats*, n.d.) (*FastAPI RepositoryStats*, n.d.)—it supports full-stack application development and powers scalable platforms used by industry leaders such as Instagram, Spotify, YouTube, Google, and NASA (Protasiewicz, 2025).

Django provides extensive built-in functionality and emphasizes speed, security, scalability, and versatility, which makes it a preferred choice among developers (Django Overview, n.d.). It supports a variety of relational databases, including PostgreSQL, MariaDB, MySQL, Oracle and SQLite, according to the official documentation (*Databases*, n.d.), while providing a powerful and intuitive ORM. More recently, Django has also introduced support for NoSQL databases such as MongoDB (*Django Integration With MongoDB Tutorial*, n.d.). Additionally, Django

includes a built-in admin dashboard that enables efficient management of CRUD operations, along with features like sorting, filtering, and more.

Support for geospatial data is built into the Django framework through **GeoDjango** (*GeoDjango Tutorial*, n.d.), which integrates with PostgreSQL via the PostGIS extension and Django **postgis backend**. Combined with Django's ORM, it offers specialized model fields, spatial queries, seamless integration with the admin interface, and a wide range of additional geospatial features.

Here is a location instance of the `Location` database table that corresponds to a specific user as displayed on the Django admin dashboard:

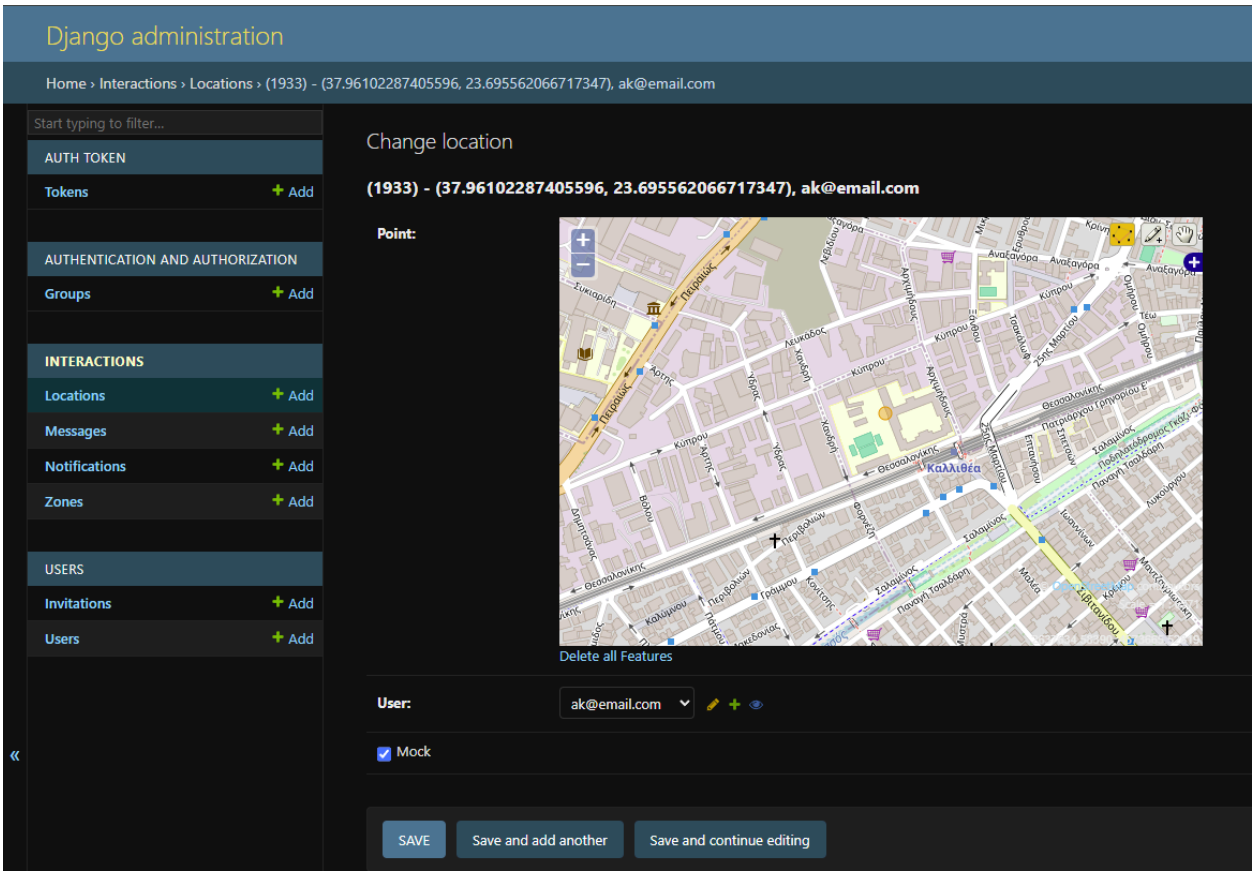

Figure 10: A user's location on the Django admin dashboard

In line with microservices architecture, the open-source **Django Rest Framework** (DRF) has become the de facto standard for building Web APIs with Django**.** DRF extends Django's capabilities by providing robust features for token-based authentication, data validation, serialization and deserialization facilitating seamless data exchange between services. When

combined with the `djangorestframework-gis` (*Djangorestframework-Gis*, n.d.) library, DRF also provides efficient support for handling geospatial data.

This can be demonstrated with a sample *GET* request to the `/api/zones/1` endpoint, which returns a zone instance defined as a GeoDjango model polygon field. The response includes the spatial data serialized in GeoJSON format.


```
{
    "id": 1,
    "type": "Feature",
    "geometry": {
        "type": "Polygon",
        "coordinates": [
            [
                [
                    23.694323,
                    37.961346
                ],
                [
                    23.694849,
                    37.960094
                ],
                [
                    23.696845,
                    37.960737
                ],
                [
                    23.696233,
                    37.961938
                ],
                [
                    23.694323,
                    37.961346
                ]
            ]
        ]
    },
    "properties": {
        "name": "zone-1",
        "created_at": "2024-12-29T19:29:05.443639Z",
        "group": 2
    }
```

```
}
```


Figure 11: A zone serialized object

A similar structure appears in a *GET* request to `/api/locations/1`, which returns a user's recorded GPS location stored as a Point field.


```
{
    "id": 1,
    "type": "Feature",
    "geometry": {
        "type": "Point",
        "coordinates": [
            23.6983667,
            37.9623023
        ]
    },
    "properties": {
        "email": "ak@email.com",
        "mock": true,
        "created_at": "2024-12-29T19:29:51.463228Z",
        "user": 1
    }
}
```


Figure 12: A user's location serialized object

As a core component of our system, the Django backend, along with Django Rest Framework, is responsible for:

- Registering and authenticating users via DRF's token authentication,
- Storing, managing and serving data to the frontend, including:
    - Users,
    - User groups,
    - Group invitations,
    - Zones defined as GeoDjango model polygon fields,
    - Notifications,
    - GPS location from the mobile application,
- Trigger Home Assistant webhooks when zone entry or exit events are detected.

4.3.4: Frontend

4.3.4.1: Streamlit Framework

Python is a language that doesn't support frontend development technologies natively. Although there are efforts to bridge this gap, such as **PyScript** (*PyScript*, n.d.), **Pyodide** (*Pyodide*, n.d.) and **Brython** (*Brython*, n.d.), Python does not run in the browser in the same way JavaScript does, nor is it used for layout, styling, or rendering like HTML and CSS.

However, frontend technologies integrate well with Python web frameworks in full-stack development. Frameworks like Django and Flask make use of tools such as Jinja HTML templates and concepts like template inheritance to help maintain a unified codebase, while keeping backend logic separate from frontend components (JavaScript, HTML, CSS).

Other frameworks like **Reflex**, **FastHTML** and **PyReact** are also full-stack solutions, but differ in that they provide Python-based abstractions over HTML and CSS. In these frameworks, HTML elements and CSS properties are represented as Python objects and constructor arguments, allowing developers to build frontend interfaces entirely in Python syntax. However, even in these cases, the developer is still responsible for writing code to define layout and styling in order to produce a usable interface.

In contrast, **Streamlit** stands out by offering strong built-in support for layout, styling, and a wide range of widgets and custom components. It allows developers to produce visually appealing results very quickly, without writing any JavaScript, HTML, or CSS, and without needing to manage those tools through wrappers. This ease of use and rapid development capability makes Streamlit an excellent choice for the needs of this thesis.

Other similar frameworks worth mentioning include **Shiny** (*Shiny for Python*, n.d.), **Panel** (*Panel - Python*, n.d.), **Gradio** (*Gradio - Python*, n.d.), **Mercury** (*Mercury - Python*, n.d.), **PyGWalker** (*PyGWalker: Turn Your Data Into Visual Analytic App*, n.d.), **Flet** (*Flet - Python*, n.d.), **NiceGUI** (*NiceGUI - Python*, n.d.) and **Taipy** (*Taipy — Build Python Data & BI Web Applications*, n.d.) among others.

4.3.4.2: Dashboard and Interaction

In what follows we will present screenshots of the user's interaction with the dashboard throughout each step of the workflow:

At first, the user can log in or register a new username and password on the login/register page. The dashboard communicates with the backend and performs user authentication using Django REST Framework (DRF) authtoken functionality:

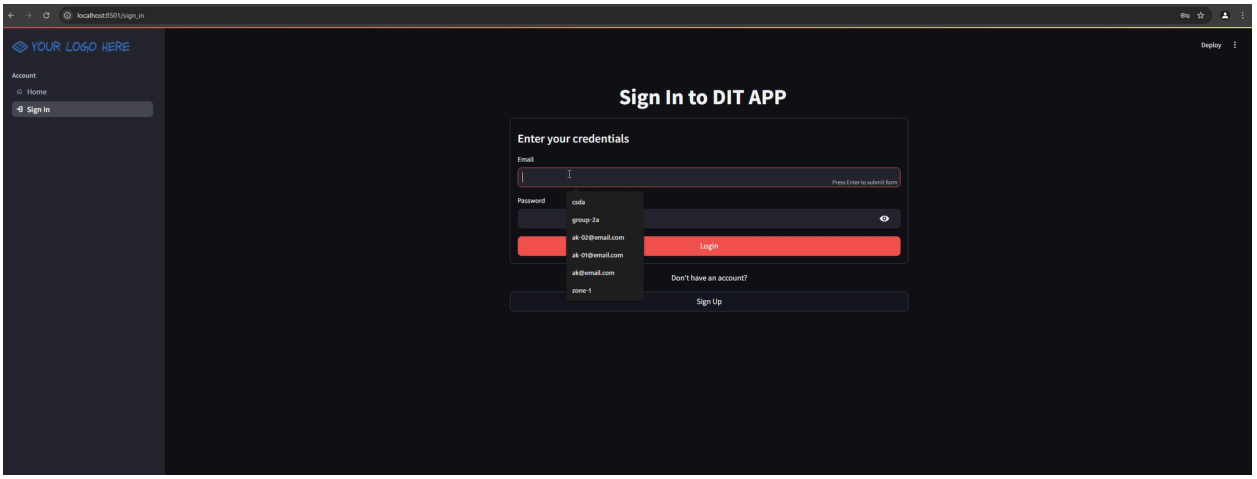

Figure 13: Dashboard - login/register page

When the user logs in, all pages available to a logged-in user are displayed in the navigation menu on the left pane. By clicking the **Management** page under the **Groups** section, the following screen is displayed, where the user can create a group by entering a name:

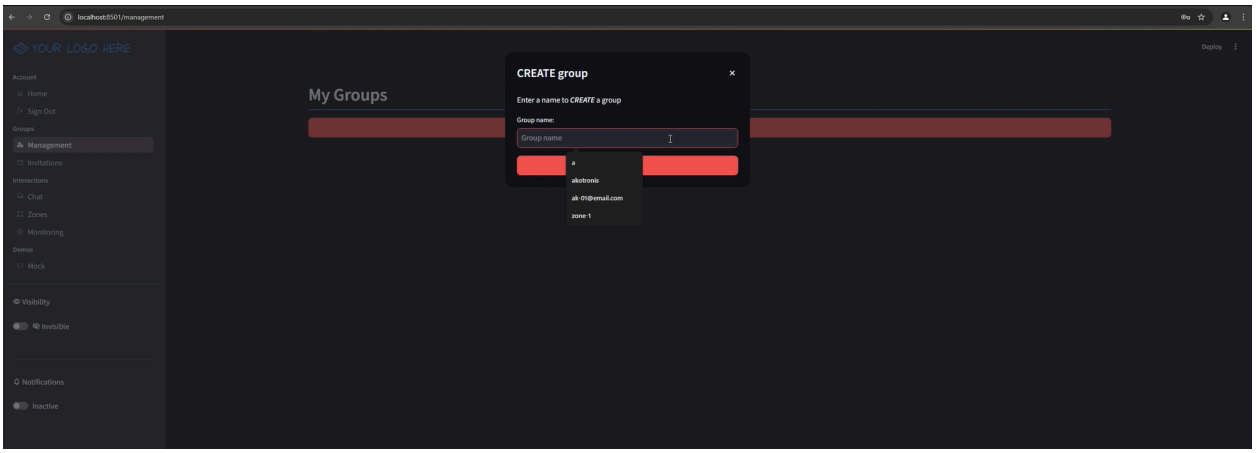

Figure 14: Dashboard - group creation

For each created group, a row is displayed in the table with the options to:

- View group info (name and members),
- Edit group details (change the name),
- Invite a user to the group by entering their email address,
- Leave a group
- Delete the group

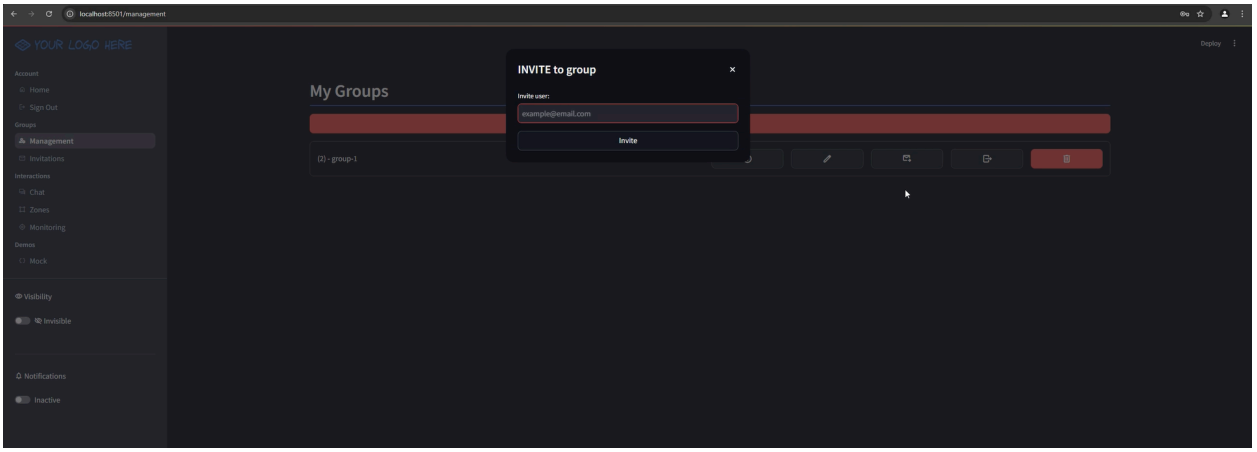

Figure 15: Dashboard - group invitation

An **Invitations** page is also available under the **Groups** section, where users can view both the invitations they have sent (along with relevant details) and those they have received. Received invitations include options to **accept** or **reject** them. By default, the status of a received invitation is **pending**:

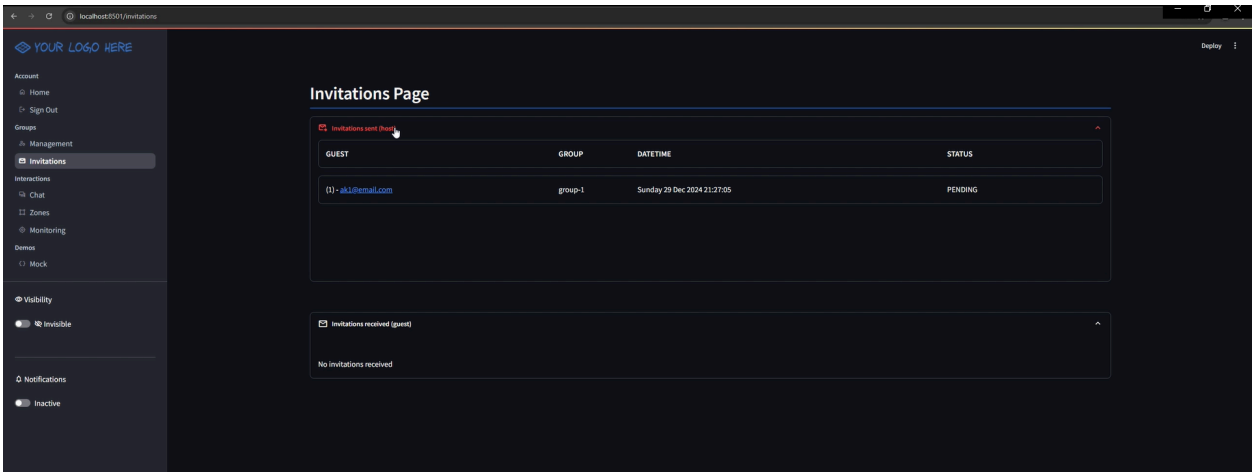

Figure 16: Dashboard - group invitations page

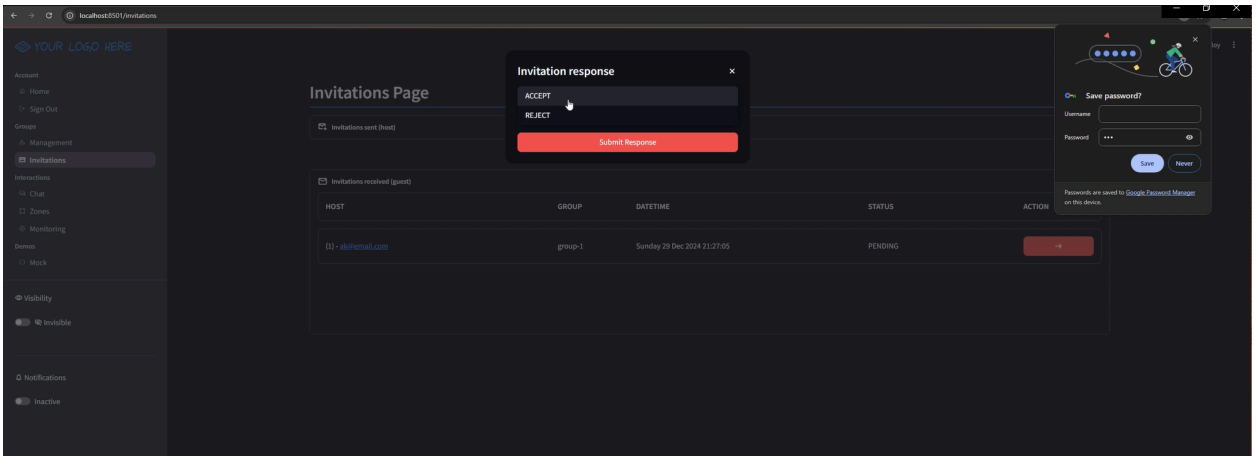

Figure 17: Dashboard - respond to group invitation

On the **Zones** page, located under the **Interactions** section, users can create a zone by providing a name, selecting a group to associate it with, and drawing a polygon area on an interactive map widget. The map widget is powered by the **streamlit-folium** package (*Streamlit-Folium*, n.d.). Existing zones can also be previewed or deleted by clicking on the **View/Delete Zones** button.

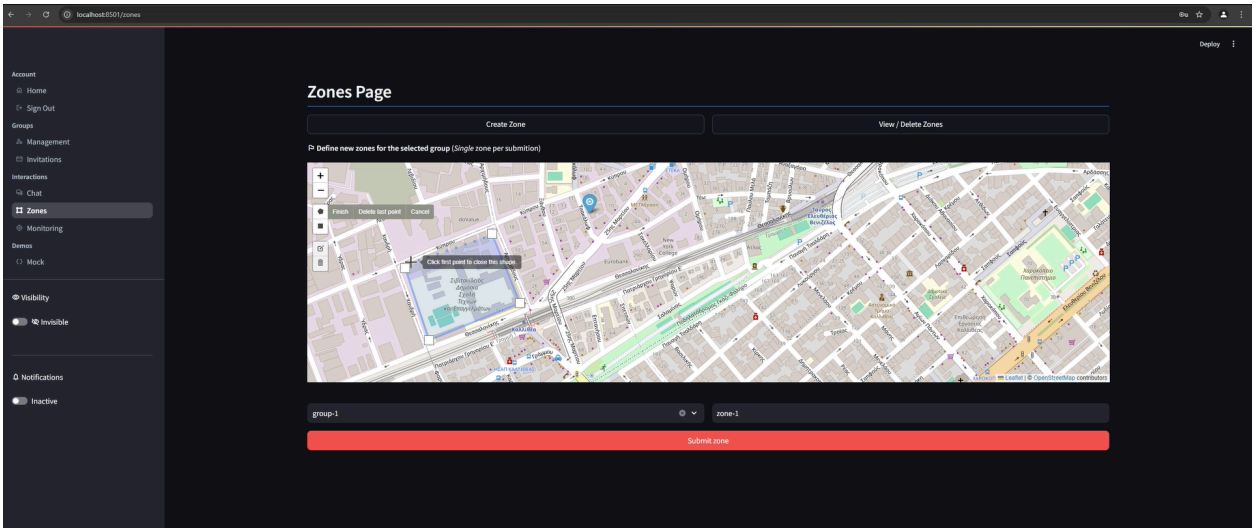

Figure 18: Dashboard - zone definition

On the **Monitoring** page, users can configure location monitoring options and submit them to start viewing user locations on the map. The configuration options include:

- A start date for the earliest location timestamp (only locations after this date are displayed),
- A group selected from those the logged-in user belongs to,

- Users within the selected group to monitor (available only in **Live monitor mode**),
- The number of latest locations to display

The **Mock monitor mode** shows locations along a predefined path for demonstration purposes, while the **Live monitor mode** displays real-time GPS data shared by mobile devices.

Additionally, the logged-in user can toggle a visibility switch in the left pane to choose whether their own location will be monitored.

If a monitored user of the selected group enters or exits a zone, a notification button is displayed on the left pane. When this button is clicked, a pop up is displayed with the relevant notification messages.

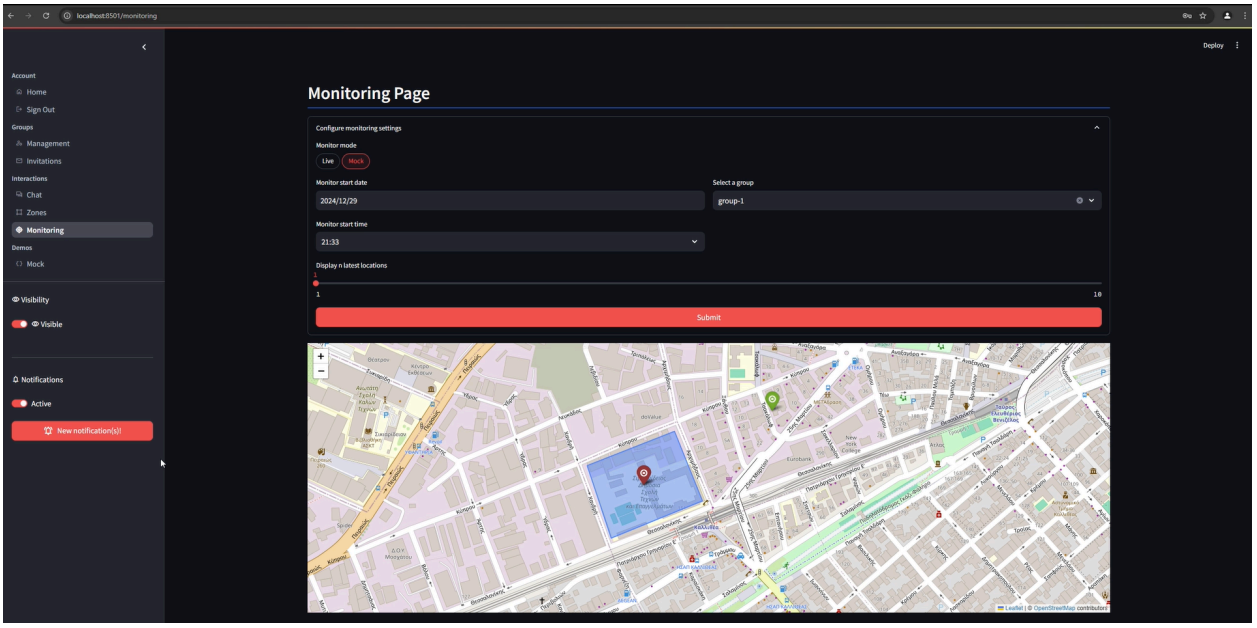

Figure 19: Dashboard - location monitoring

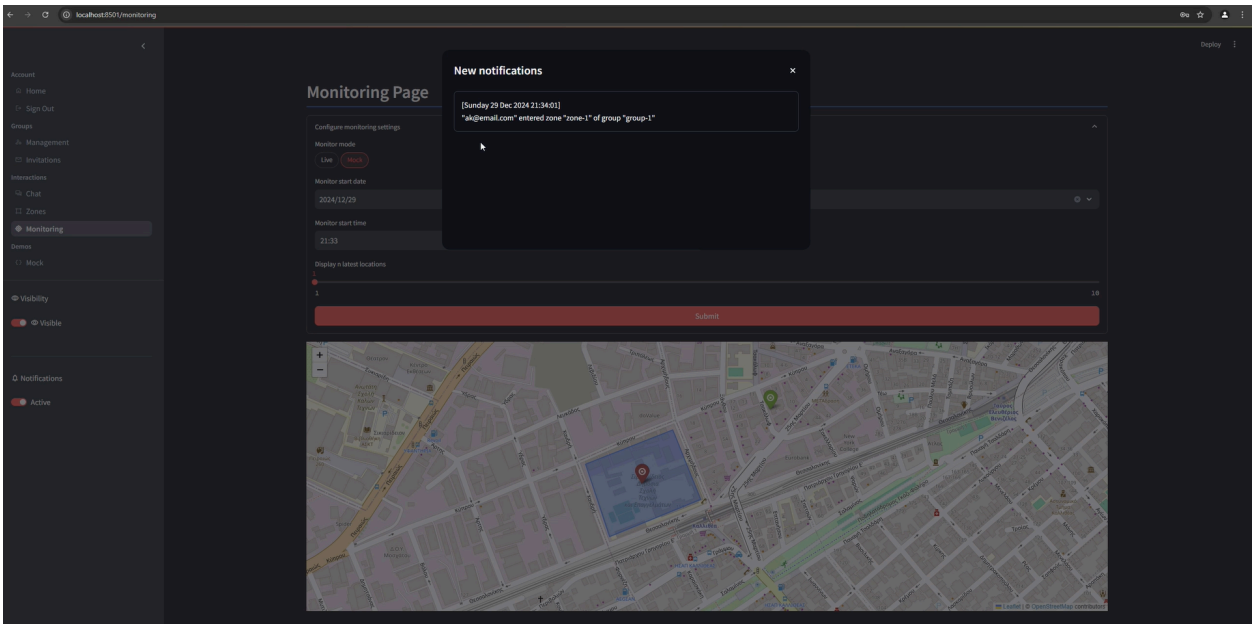

Figure 20: Dashboard - notifications display popup

4.3.5: Mobile Application

4.3.5.1: Kivy Framework and KivyMD

Kivy is a community-driven open source framework released under the MIT Licence (Golubic, 2023). It is maintained by contributors, supported by backers, and enables the development of applications for both Android and iOS using Python as its programming language.

While Kivy is not as widely adopted as other mobile development frameworks, it remains popular within the Python community due to its simplicity and ease of use. The fact that it continues to be actively maintained more than 14 years after its initial release in 2011 indicates that it still serves a practical purpose and retains an engaged user base (*Kivy · PyPI*, n.d.).

At the core of Kivy's features is support for the **Kv language**, which is used to define user interfaces and map events to callbacks in a declarative style. This approach promotes separation of concerns between interface design and business logic. In projects built with the Kivy framework, Kv language typically resides in separate files with the *.kv* extension.

Some of the reason Kivy might be a preferred choice for development include:

- **Speed**, as it is optimized with implemented functionality on C,

- **Flexibility**, as it can run on many devices and supports all major operating systems such as Windows, Linux, macOS and BSD,
- **Simplicity,** as it offers a focused, developer-friendly environment that emphasizes building functional applications quickly, supported by the use of Kv language.

(Kivy Docs - Philosophy, n.d.)

By design, Kivy uses its own widgets and rendering system that provides flexibility at the cost of a native look and feel. It typically has the same interface across devices and does not match the native style users might expect from their device. **KivyMD** addresses this by providing a collection of Material Design-compliant widgets that bring a more familiar and professional appearance to Kivy applications, especially on Android and desktop platforms (Rodríguez & Bulgakov, n.d.).

4.3.5.2: Interface and Interaction

Once the application is launched on the mobile device, the user interface is displayed. By tapping the menu button on the top left, the user can toggle the navigation drawer, which includes access to the **Home**, **Log In**, and **Sign Up** pages, as well as the **Toggle theme** and **Share location** buttons which are currently inactive.

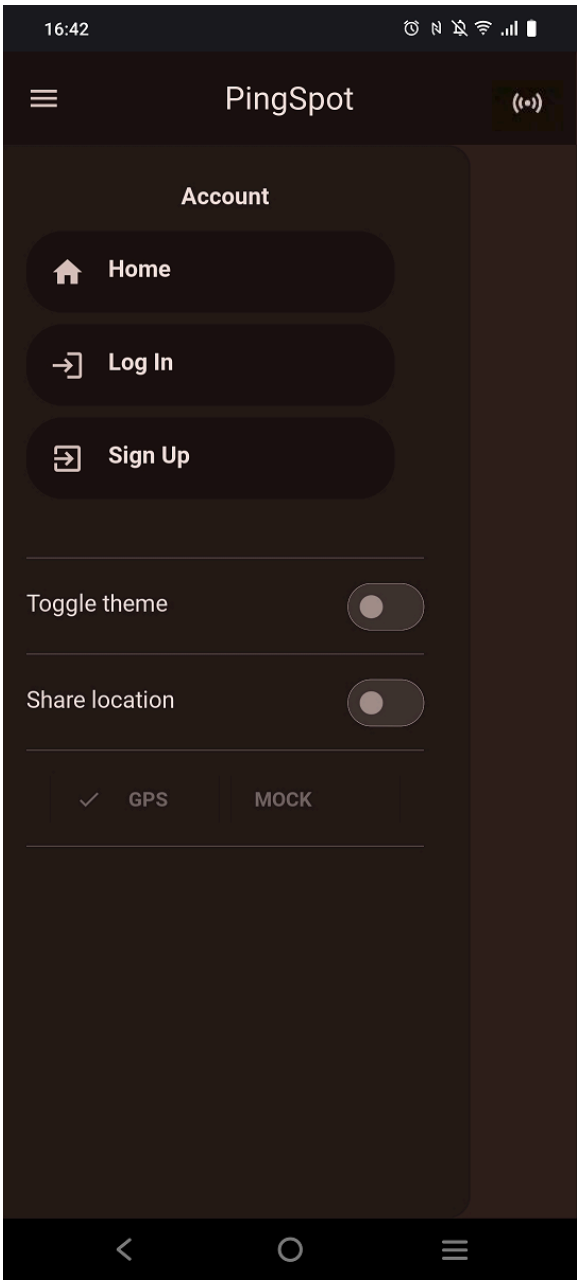

Figure 21: Mobile - Navigation drawer before established connection

The user can freely navigate between pages via the navigation drawer. However, in order to log in or register and begin sending location data to the backend, a connection between the mobile app and the backend service must first be established. This is initiated by tapping the **connection icon** at the top right of the screen.

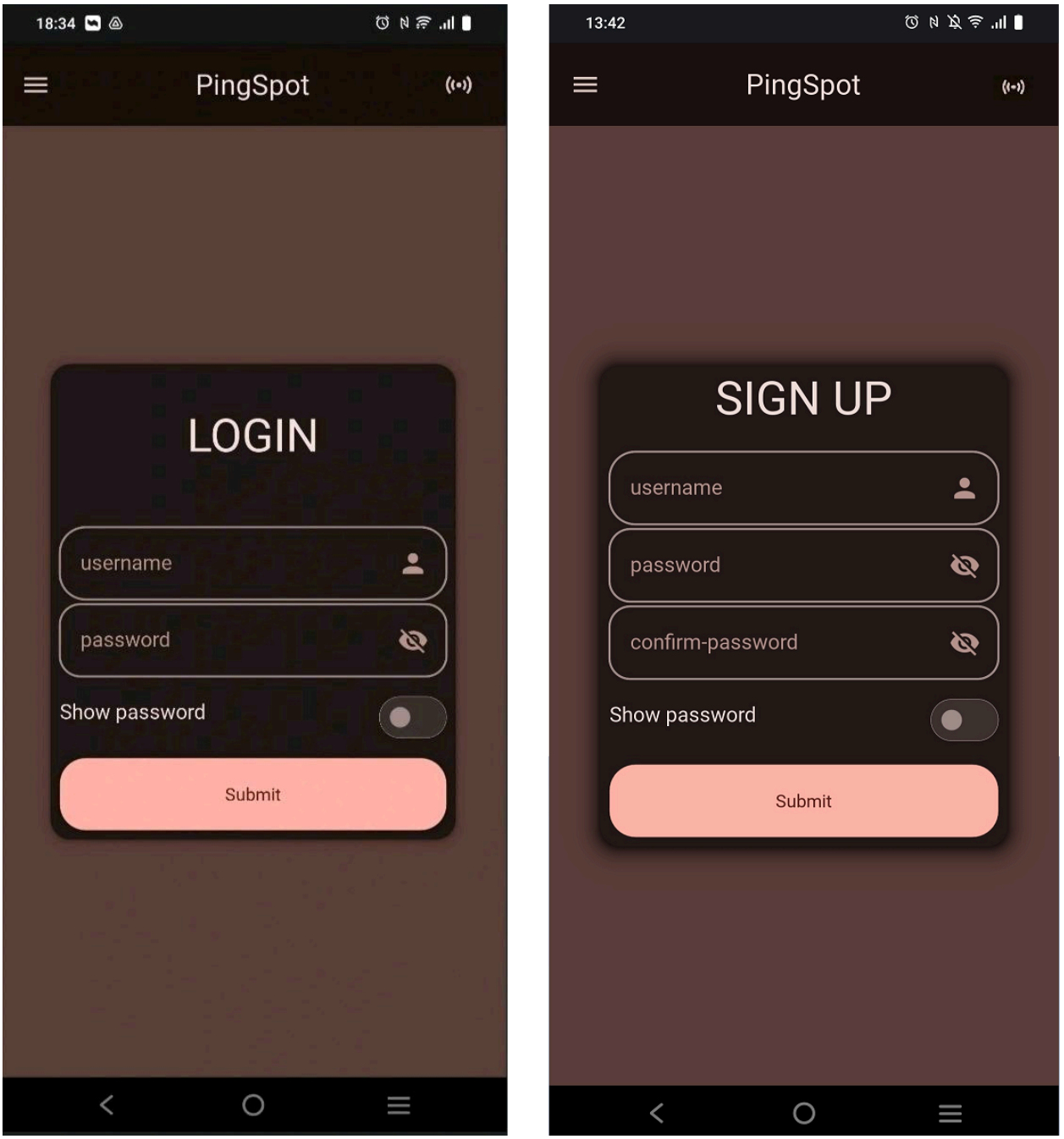

Figure 22: Mobile - Login and sign up mobile application screens

To enable this connection, a VPN tunnel must first be created through the WireGuard mobile application. After installing WireGuard from Google Play, the user can easily set up the tunnel by importing the provided client configuration files. The following screens illustrate the tunnel setup process and connection details:

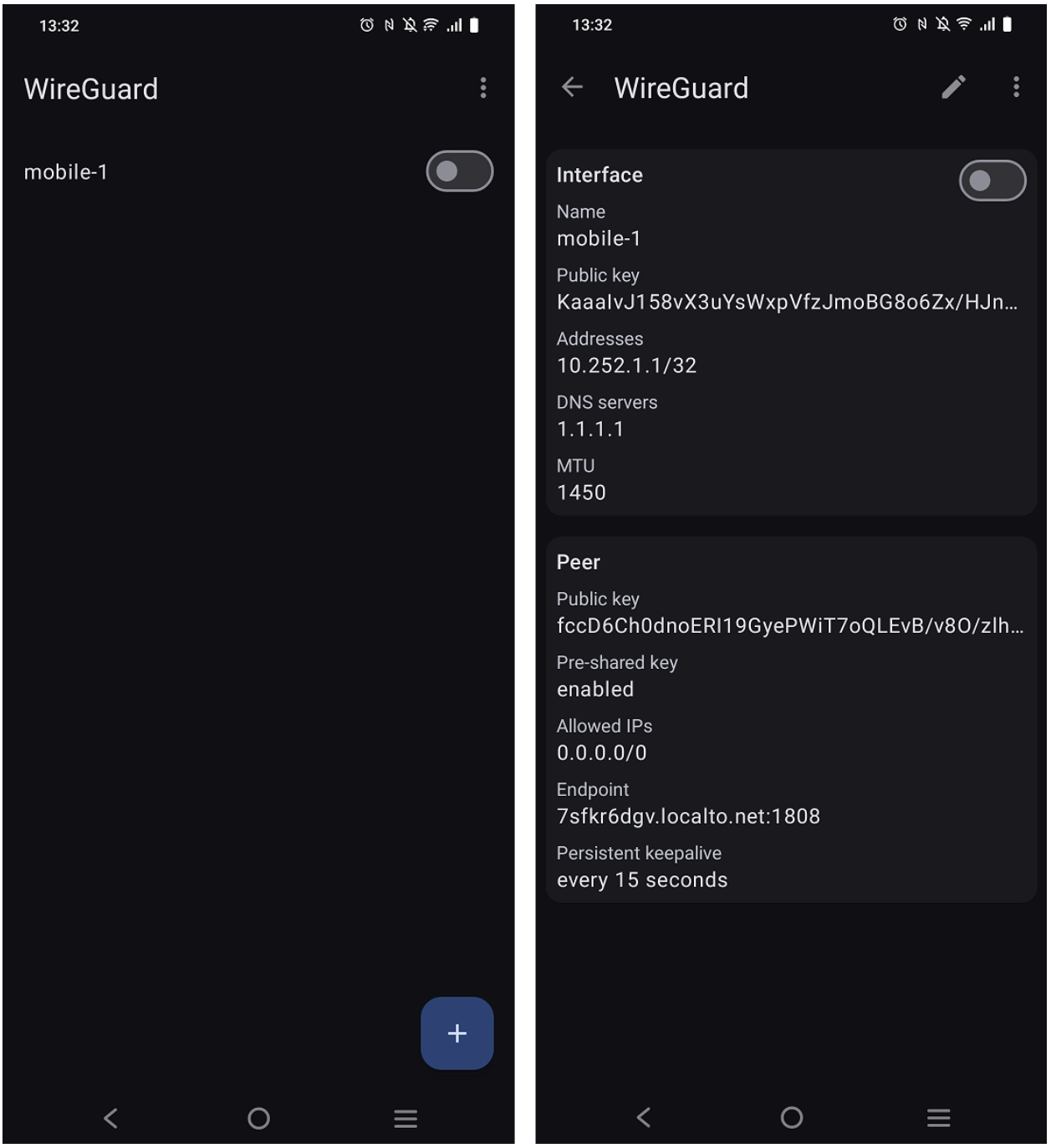

Figure 23: WireGuard mobile application interface

If the mobile application cannot connect to the backend service, the user is notified with an appropriate error message. If the backend is reachable, the user is informed accordingly and can proceed to log in or register. Upon successful login, several UI changes occur: a user icon appears on the left side of the top bar, the connection button changes color, and all pages accessible to authorized users become visible on the navigation drawer. Additionally, switch buttons and location-sharing options are enabled. An active VPN tunnel created via the WireGuard mobile app is indicated by a key icon at the top of the screen.

The user can now switch between light and dark themes, enable or disable location sharing, and choose whether to share real-time GPS data or mock location data following a predefined path for demonstration purposes.

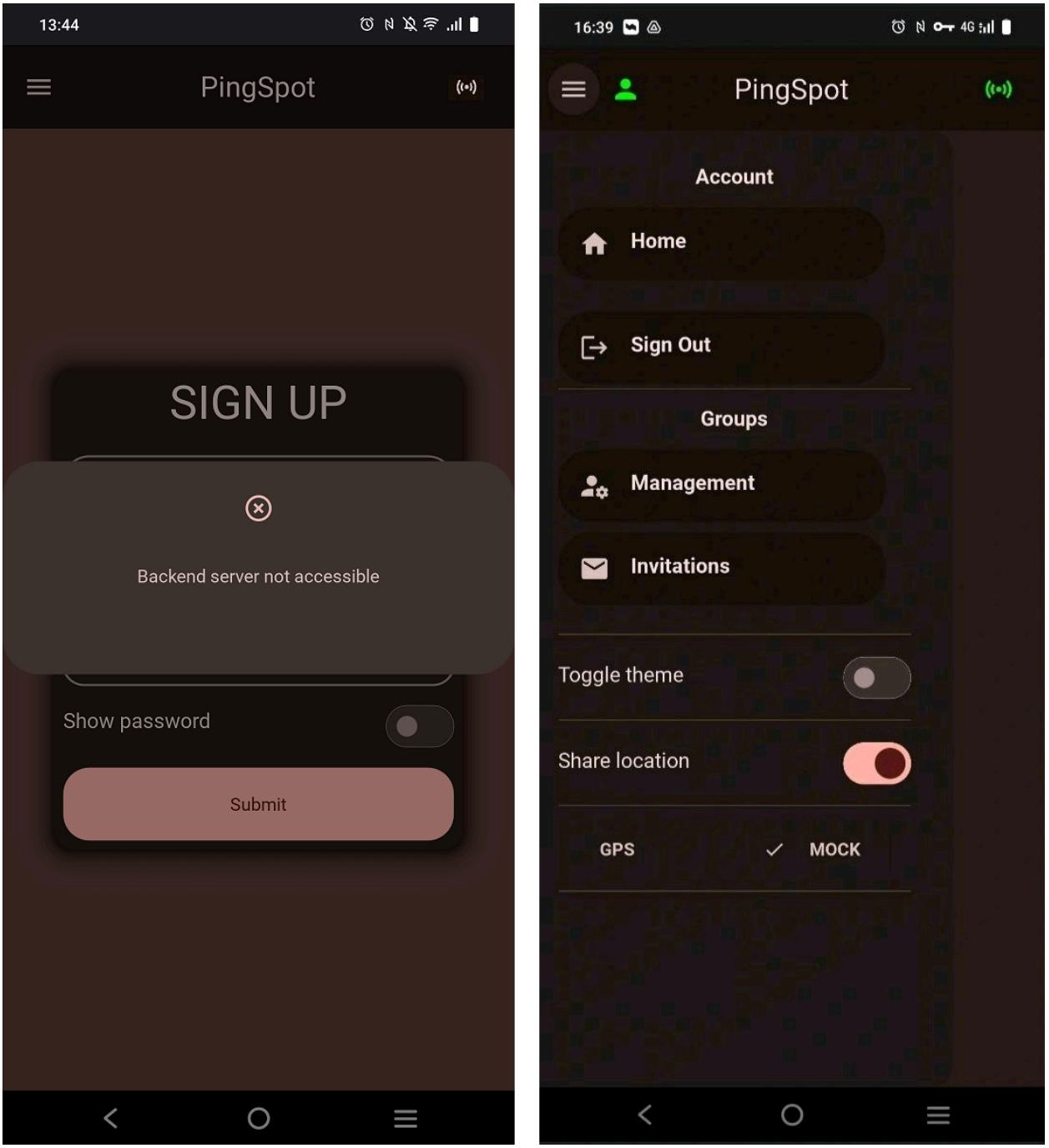

Figure 24: Mobile - Pop-up messages and user interface after login

4.3.6: Home Assistant

Home Assistant is a popular, widely used, free and open-source home automation platform. It serves as a centralized hub for controlling, automating, and monitoring smart home devices without relying on cloud services, offering enhanced security, customization, and reliability. Home Assistant can run on various systems such as a Raspberry Pi, a server, or a virtual machine. It offers a user-friendly interface for management and configuration, and is highly extensible through add-ons and custom integrations.

The above characteristics make Home Assistant a suitable choice as a component of our system. Specifically, it is responsible for connecting with the Tapo smart plug and defining webhook endpoints to toggle the device's power on and off. These webhooks are triggered by the Django backend service when a user enters or exits predefined zones.

Below are screenshots showing the device registration in Home Assistant, the webhook configuration, and the toggling of the Tapo smart plug's power supply upon webhook triggers.

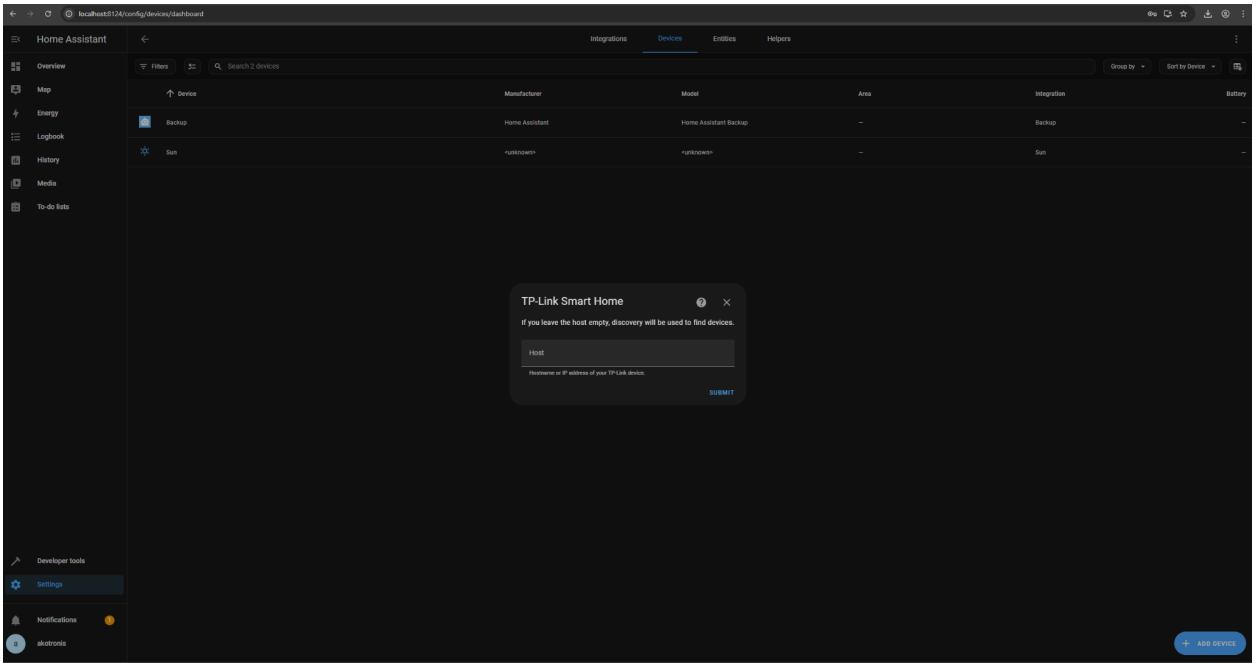

Figure 25: Register device on Home Assistant

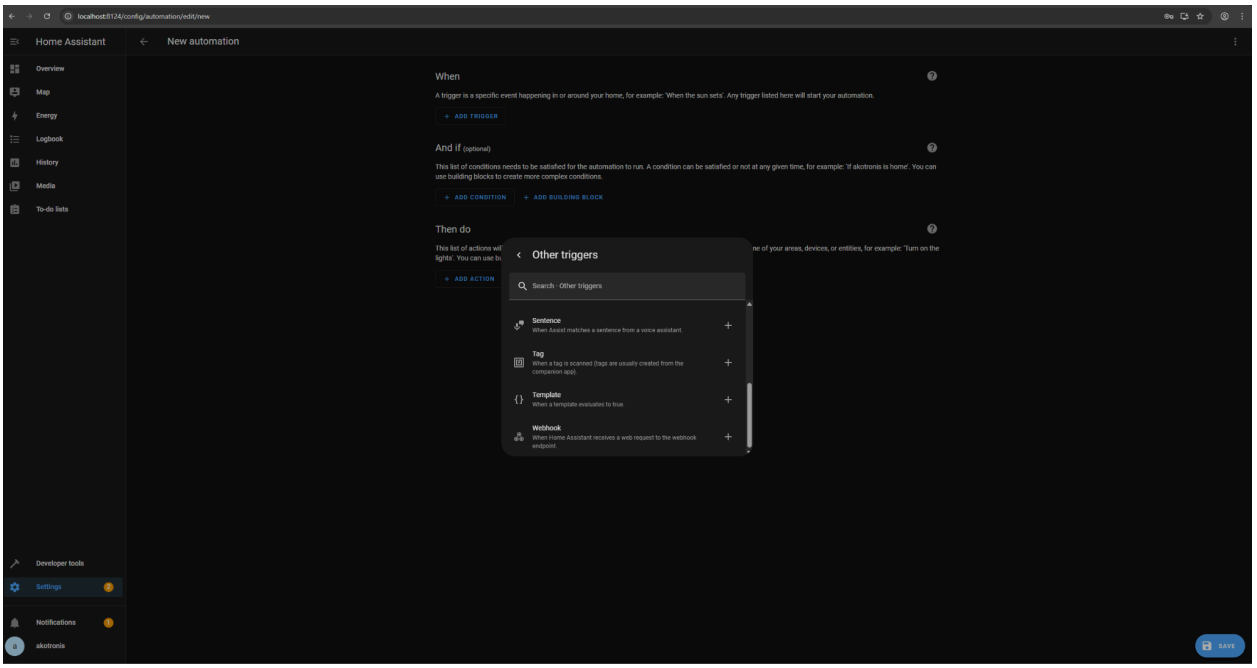

Figure 26: Webhook configuration on Home Assistant

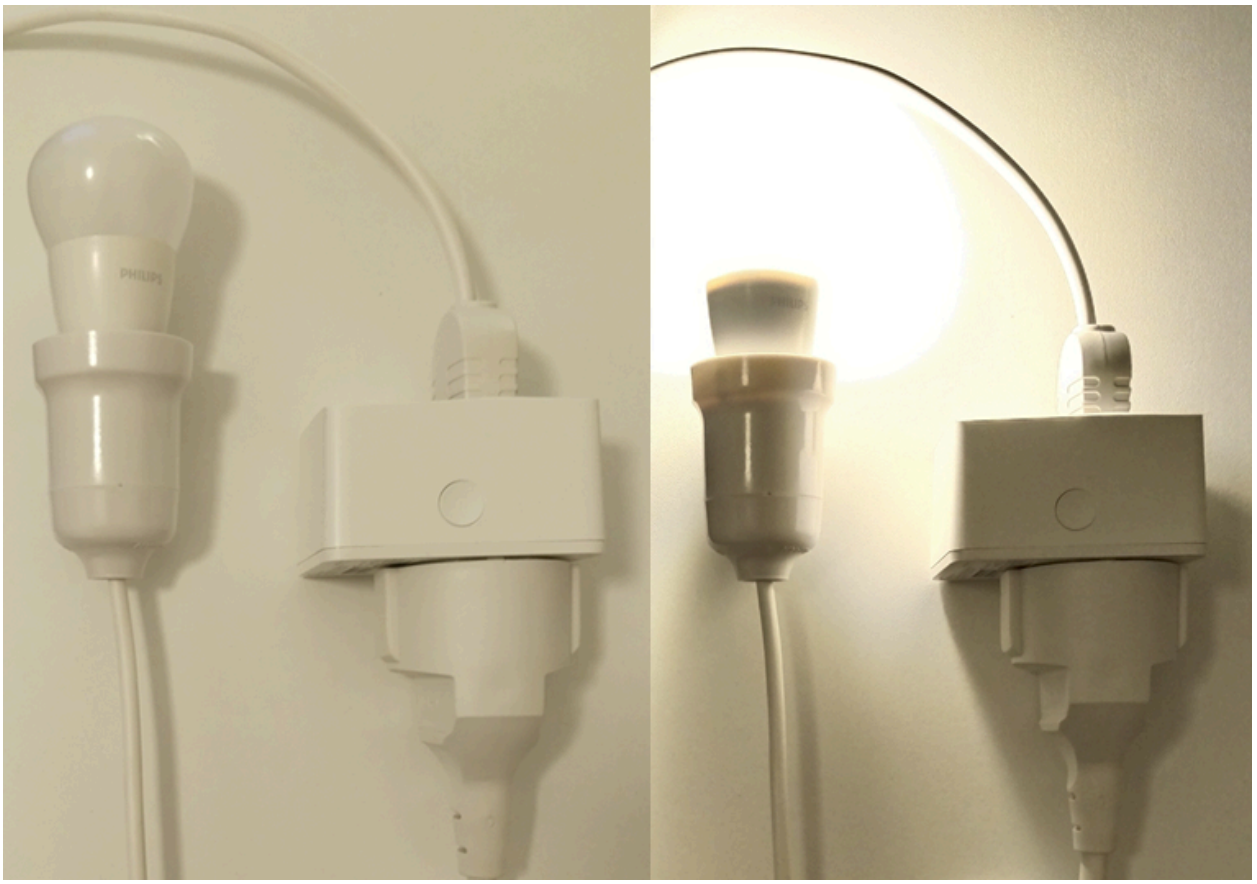

Figure 27: Tapo power supply toggling on webhook triggers

4.3.7: Virtual Machine Development Environment

A critical aspect of development is the choice of the environment. There are no strict rules or one-size-fits-all solutions for this. The decision depends on several factors specific to each case. For instance, if development is carried out by a team rather than an individual, **consistency** becomes an important consideration. The application should run without requiring separate setup and configuration on each developer's machine to avoid bottlenecks during the development process.

Developers may use different operating systems, such as Linux, Windows, or macOS, so the chosen development environment should offer a unified solution that is independent of individual preferences. Another factor influencing this decision is the set of tools used in the development process. Some tools run natively only on specific operating systems and require additional configuration to work elsewhere.

In many such cases, a **virtual machine** provides a practical solution. A virtual machine (VM), as the name suggests, is a virtualized computer that runs its own operating system in an isolated environment (*What Is a Virtual Machine?*, n.d.). For example, a VM can offer a consistent Linux environment for all team members, regardless of the host operating system they use.

There are many tools available for setting up and managing virtual machines. In our case we use **Vagrant** in combination with **VirtualBox.** According to its official documentation, Vagrant is a command-line tool used to manage the lifecycle of virtual machines. It uses special configuration files called **Vagrantfiles** which define the virtual machine's settings and can also include software installation steps. This allows the entire environment setup process to be tracked, automated, and consistently reproduced.

To set up virtual machines, Vagrant relies on an underlying virtualization technology offered by *providers*. Vagrant uses **VirtualBox** as its default provider.

In our case, the decision to use a virtual machine as the development environment was driven by issues encountered during the Home Assistant setup and device discovery on a Windows

system, compared to the smoother experience in a Linux environment provided by Vagrant and VirtualBox.

Below is the script for the virtual machine configuration used for our workflow:

```
# -*- mode: ruby -*-
# vi: set ft=ruby :

Vagrant.configure("2") do |config|
    config.vm.box = "generic/ubuntu2204"
    config.vm.synced_folder "./homeassistant",
"/home/vagrant/code/DIT-thesis/app/ubuntu/homeassistant"
    config.ssh.insert_key = true
    # Use a bridged (public) network adapter as well as NAT (Default
adapter 1) with a fixed ip
    config.vm.network "public_network", ip: "192.168.1.17"
    config.vm.network "forwarded_port", guest: 8123, host: 8123

    config.vm.provider "virtualbox" do |vb|
      vb.name = "DIT-Thesis"
      vb.gui = false
      vb.memory = 4096
      vb.cpus = 2
    end
  end
```

Figure 28: Virtual machine configuration with Vagrant

4.3.8: End-to-end flow

In this section we present all the necessary steps for a complete, working example of our system's workflow, including configuration and end-user actions:

4.3.8.1: Localtonet

First we log in to our localtonet account and navigate to the **TCP/UDP Tunnel configuration** section on the dashboard by visiting `https://localtonet.com/tunnel/tcpudp`. In

the tunnel settings, we ensure that the local IP address of the **WireGuard** container and the corresponding port are correctly added. As explained in the **Implementation** section below, the **WireGuard**, **Django backend**, and **Streamlit frontend** containers are configured with fixed IP addresses. This ensures that the WireGuard container's IP does not need to be updated repeatedly on the localtonet dashboard, and that the mobile device can consistently access the backend and frontend services.

Finally, we click the **Play** button to launch the tunnel.

4.3.8.2: Ngrok

We navigate to the **Domains** section of our ngrok account by visiting `https://dashboard.ngrok.com/domains` and make sure that the required domain pointing to the WireGuard UI is defined. The URL provided by ngrok has to be included in the Docker Compose definition of the local ngrok service as will be explained in the **Implementation** section below.

4.3.8.3: WireGuard Configuration

- Before we proceed with the WireGuard configuration, we:
    - Launch the virtual machine by running `$ vagrant up` in the directory where the Vagrantfile file is located.
    - SSH into the virtual machine by running `$ vagrant ssh` from the same directory.
    - Start the local services by running `$ docker compose up -d` inside the virtual machine, in the directory where the docker compose configuration file is located.
- We log in to the WireGuard UI, which is accessible from the browser on `http://localhost:5000/global-settings`, with the predefined administrator credentials.
- On the **Endpoint Address** under the **Global Settings** sections we add the **server domain** provided by localtonet and **Save**. Note that when the tunnel stops and restarts the port changes of the server domain changes to a new one.
- On **WireGuard Server Settings** we must have:

- The WireGuard service's port on the **Listen Port** textbox.
- On **Post Up Script:** `iptables -t nat -A POSTROUTING -s 10.252.1.0/24 -d 172.28.1.0/24 -j MASQUERADE; iptables -A FORWARD -i wg0 -d 172.28.1.0/24 -j ACCEPT; iptables -A FORWARD -s 172.28.1.0/24 -o wg0 -j ACCEPT; iptables -A INPUT -p udp -m udp --dport 51820 -j ACCEPT; iptables -A FORWARD -i wg0 -j ACCEPT; iptables -A FORWARD -o wg0 -j ACCEPT;`
- On **Post Down Script:** `iptables -t nat -D POSTROUTING -s 10.252.1.0/24 -d 172.28.1.0/24 -j MASQUERADE; iptables -D FORWARD -i wg0 -d 172.28.1.0/24 -j ACCEPT; iptables -D FORWARD -s 172.28.1.0/24 -o wg0 -j ACCEPT; iptables -D INPUT -p udp -m udp --dport 51820 -j ACCEPT; iptables -D FORWARD -i wg0 -j ACCEPT; iptables -D FORWARD -o wg0 -j ACCEPT;`

  where `172.28.1.0/24` is the subnet defined in the docker compose network configuration. The **Post Up** and **Post Down** scripts are required in order to route incoming traffic from the localtonet tunnel to the backend services.

- With the above configuration saved we click the **Apply Config** button at the top of the WireGuard UI interface.
- For each user whose location will be monitored, an account can be created from the WireGuard UI. This account can then be used on the mobile device to generate the corresponding client configuration. Alternatively, client configurations can be created manually, and the resulting configuration files can be sent to users via email, allowing them to import the settings and share their location. We will use the first approach.

4.3.8.4: Mobile client

With the WireGuard service's Global and Server settings configured, we now explore the steps required on the **mobile side** by the device owner to enable location sharing.

- The **WireGuard** mobile application must be downloaded from **Google Play** and installed on each device that will share its location along with the **Kivy application**.

- The user opens the browser, visits the **WireGuard UI** using the link provided on the **ngrok dashboard**, and logs in with their credentials.
- They click the **New Client** button, enter a name, and then click **Save**.
- From the **WireGuard Clients** section, the client configuration can be exported as a *.conf* file.
- In the **WireGuard mobile app**, the user clicks the **+** button to create a new tunnel and imports the *.conf* file.
- Next, they open the **Kivy application**, check the backend server's accessibility by clicking the **connection icon** at the top-right corner, and wait for a confirmation popup.
- If the popup confirms that the backend server is accessible, the user can either **log in** or **register**, and then enable the **Share Location** switch.
- By selecting **GPS** as the location source, the application begins sending the device's GPS location to the backend service.

4.3.8.5: Monitoring locations from the dashboard

Assume that a user with the email `user-1@email.com` wants to track the location of another user, `user-2@email.com`, from the dashboard. The following steps are required:

- **Both users must be registered** in the system. They can complete registration either from the dashboard or the mobile application.
- `user-1@email.com` logs into the dashboard, creates a group and invites `user-2@email.com` to join that group.
- `user-2@email.com` must log into the dashboard and **accept the invitation** in order to become a member of the same group.
- For `user-1@email.com` to monitor the location of `user-2@email.com`:
    - `user-2@email.com` must complete the required setup steps to **share their GPS location** as described earlier.
    - `user-1@email.com` then visits the **Monitoring** page on the dashboard and selects `user-2@email.com` under the appropriate group.

Once these steps are completed, the **GPS location of** `user-2@email.com` will appear as a marker on the dashboard map, provided the **Share Location** switch is enabled on their mobile device.

4.3.8.6: Geofencing and notifications preview from the dashboard

The geofencing and notifications feature of the system requires the following steps from the user:

- The user must define a **zone** from the **Zones** page of the dashboard and associate it with one of the groups they belong to.
- The **Notifications** switch on the left pane must be enabled.

Once these actions are completed, the user can view location markers on the map in the **Monitoring** page based on their group and user selections. When a monitored user **enters or exits** a defined zone, a **New Notifications** button will appear, and clicking it will display the relevant information in a popup window.

4.3.8.7: Home Assistant and Event Triggers

The steps required to register the Tapo smart plug and define webhooks in Home Assistant are listed below:

- **Device Registration**
    - Get the **Tapo device IP** from the router web interface under *Allocated Address (DHCP)* section
    - Run `$ ping <tapo-ip-address>` or `$ curl http://<tapo-ip-address>:80` inside the VM to ensure the device is accessible
    - In the Home Assistant user interface:
        - Go to *Settings* on the left pane
        - Click on *Device & Settings*
        - Select *Device* Tab
        - Click the *Add Device* button on the bottom right
        - Search for *Tp-Link*
        - Select *Tapo*

        - Enter the Tapo device IP and submit
- **Webhook definition.** In the Home Assistant user interface:
    - Go to *Settings* on the left pane
    - Click on *Automations and Scenes*
    - Click on *Create Automation*
    - Click on *Create new automation*
    - Click on *Add a trigger*
    - Search for *Webhook* and add ID
    - Select *Add action*
    - Select the Tapo device and action *turn on*
    - Save
    - Do the same for *turn off*
    - The webhook IDs defined will be used to control the Tapo plug via POST requests to the following endpoints:
        - `api/webhook/<tapo-webhook-turn-on-id>`
        - `api/webhook/<tapo-webhook-turn-off-id>`

Chapter 5: Implementation

In this chapter, we take a closer look at the technical aspects of the system's implementation and outline some of the details of the setup, configuration, and integration of its various components. Through selected examples and explanations, we highlight key implementation decisions and offer insight into the development process.

5.1: Dockerized Services, Networking and Orchestration

In Chapter 4, we mentioned that the system was developed within a Linux virtual machine and provided the corresponding Vagrantfile configuration. Since this VM serves as the host for the system's services, which are orchestrated using Docker and Docker Compose, these tools must be installed. The following steps outline how to install Docker and Docker Compose inside the VM, either manually, or as provisioning steps within the Vagrantfile, following the procedures described in the official documentation for Docker Engine and Docker Compose (*Install Docker Engine on Ubuntu*, n.d.) (*Install the Docker Compose Plugin*, n.d.).

Docker installation on Ubuntu:

1. Uninstall all conflicting packages:

```
- $ for pkg in docker.io docker-doc docker-compose docker-compose-v2
  podman-docker containerd runc; do sudo apt-get remove $pkg; done
```

2. Install using the apt repository:

```
- $ sudo apt-get update
- $ sudo apt-get install ca-certificates curl
- $ sudo install -m 0755 -d /etc/apt/keyrings
- $ sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o
  /etc/apt/keyrings/docker.asc
- $ sudo chmod a+r /etc/apt/keyrings/docker.asc
- $ sudo -i
- # apt update
- # apt install sudo
- # exit
- $ sudo --version
```

```
- $ echo "deb [arch=$(dpkg --print-architecture)
  signed-by=/etc/apt/keyrings/docker.asc]
  https://download.docker.com/linux/ubuntu $(. /etc/os-release &&
  echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}") stable" | sudo tee
  /etc/apt/sources.list.d/docker.list > /dev/null
- $ sudo apt-get update
- $ sudo apt-get install docker-ce docker-ce-cli containerd.io
  docker-buildx-plugin docker-compose-plugin
- $ docker --version
- $ sudo usermod -aG docker $USER
- $ newgrp docker
```

Docker Compose installation on Ubuntu:

1. Install and verify installation:

```
- $ sudo apt-get update
- $ sudo apt-get install docker-compose-plugin
- $ docker compose version
```

Another critical aspect of networking the system components, as discussed in Chapter 4, is ensuring that the WireGuard, backend, and frontend services have fixed IP addresses. This is necessary because the WireGuard service's IP must be registered in the Localtonet dashboard, and we want to avoid it changing every time the service is launched. Additionally, fixed IPs allow the mobile device to reliably access the other services, particularly the frontend, and optionally the backend, which offers added control and flexibility. To achieve this, we can define a custom network in the Docker Compose configuration file as follows:

```
networks:
  ntw-local:
    driver: bridge
    ipam:
      config:
        - subnet: 172.28.1.0/24
```

Figure 29: Docker compose network definition

and add this network on each of the three mentioned services definition with their corresponding fixed IP:

```
services:
  srv-dit-wireguard:
    networks:
      ntw-local:
        ipv4_address: ${WIREGUARD_FIXED_IP}

  srv-dit-back:
    networks:
      ntw-local:
        ipv4_address: ${BACKEND_FIXED_IP}

  srv-dit-front:
    networks:
      ntw-local:
        ipv4_address: ${FRONTEND_FIXED_IP}
```

Figure 30: Fixed IPs on service definitions

Localtonet and ngrok reverse proxies provide dedicated templates for defining local Docker services. These services act as bridges between the local system and their corresponding remote endpoints, enabling external access to the local environment.

Below are the service definitions for the reverse proxies. The connection between the local WireGuard UI and the ngrok platform is established by referencing the local WireGuard UI port, 5000, and the endpoint provided by ngrok: `grand-implicitly-escargot.ngrok-free.app`.

```
  srv-dit-localtonet:
    image: localtonet/localtonet:latest
    container_name: ctr-dit-localtonet
    command: authtoken ${LOCALTONET_AUTHTOKEN}
    restart: unless-stopped
    networks:
      - ntw-local

  srv-dit-ngrok:
    image: ngrok/ngrok:3
    container_name: ctr-dit-ngrok
```

```
          command:   http   --url=grand-implicitly-escargot.ngrok-free.app
ctr-dit-wireguard:5000
    environment:
      NGROK_AUTHTOKEN: ${NGROK_AUTHTOKEN}
    ports:
      - 4040:4040
    restart: always
    networks:
      - ntw-local
```

Figure 31: Reverse proxy local service definitions

The setup of Home Assistant, including its networking configuration and integration with other system components, involves several subtleties. The official documentation provides a template for a docker compose installation (*Home Assistant Installation*, n.d.), so with this in mind we used the following definition:

```
  srv-dit-homeassistant:
    image: "ghcr.io/home-assistant/home-assistant:stable"
    container_name: ctr-dit-homeassistant
    volumes:
      - dit-homeassistant-data:/config
      - /etc/localtime:/etc/localtime:ro
      - /run/dbus:/run/dbus:ro
    restart: unless-stopped
    privileged: true
    environment:
          -  HOMEASSISTANT_OPTS=--config  /config  --log-file  /dev/stdout
--log-level debug
    network_mode: host
```

Figure 32: Home Assistant service definition

With the above configuration, a specific networking challenge arises: the `network_mode: host` option places the Home Assistant service directly on the host network stack, i.e., the virtual machine. We need to access Home Assistant in two distinct scenarios:

1. From a web browser to pair the smart plug and configure automation triggers
2. From the backend service to programmatically trigger actions when geofencing events occur.

There are two potential approaches to handle this: a *static* and a *dynamic* one:

1. **Static approach**: Launch the VM with a fixed IP address (as described in the Vagrantfile in Section 4.3.7) and:
    - Use **the VM fixed IP** with the port Home Assistant is running inside the VM, or **`localhost` with the forwarded port**, to access Home Assistant from the browser.
    - Use **the VM fixed IP** with the port Home Assistant is running inside the VM, or the **`extra_hosts`** option, as described below, to access Home Assistant from the backend service.
2. **Dynamic approach**: Launch the VM without the fixed IP described in the Vagrantfile in Section 4.3.7 and:
    - Use **`localhost` with the forwarded port**, to access Home Assistant from the browser.
    - Configure the backend service using the **`extra_hosts`** option and the **`host.docker.internal:host-gateway`** mapping. This allows the container to resolve the host's IP dynamically and access Home Assistant using `host.docker.internal:<home-assistant-port>` from within the container (Janetakis, 2024).

Here is the backend docker compose configuration with the **`extra_hosts`** option that works with both approaches:

```
srv-dit-back:
  build: ./back
  container_name: ctr-dit-back
  restart: always
  ports:
    - ${BACKEND_EXTERNAL_PORT}:${BACKEND_INTERNAL_PORT}
  volumes:
    - ./back:/opt/back
  env_file:
    - .env
  environment:
    - POSTGRES_HOST=srv-dit-db
    - VM_FIXED_IP=${VM_FIXED_IP}
  depends_on:
    srv-dit-db:
      condition: service_healthy
```

```
    networks:
      ntw-local:
        ipv4_address: ${BACKEND_FIXED_IP}
    extra_hosts:
      - "host.docker.internal:host-gateway"
```

Figure 33: Backend service definition

It should be noted, however, that with the *dynamic* approach, the Tapo device **is not accessible** from within the VM, and thus not accessible to Home Assistant, because the VM is isolated from the local home network where the device is connected. Therefore, the static approach with a fixed IP is necessary.

5.2: Mobile Application

One of the key aspects in developing the mobile application was managing dependencies locally during development and ensuring they were properly integrated into the build process.

For development, dependency management was handled using *Astral's* **uv** (*Uv*, n.d.), a modern Python package and project manager written in Rust.

uv offers several important features, including high performance, consistent dependency resolution, efficient caching, compatibility with the **pip** interface, Python version management, workspace support, and more. These capabilities have made it a preferred tool among Python developers today. It manages dependencies through *uv.lock* and *pyproject.toml* files, following the standards introduced in **PEP 518** (*Python PEP 518*, n.d.). The *pyproject.toml* file defines project metadata and declared dependencies, while the *uv.lock* file records the exact versions of all resolved packages. This format is similar to the **Poetry** package manager, which uv aims to replace, along with other legacy tools.

Here is the *pyproject.toml* file used for the mobile application development:

```
[project]
name = "mobile"
version = "0.1.0"
description = "Add your description here"
```

```
requires-python = ">=3.11"
dependencies = [
    "kivy==2.3.1",
    "kivymd",
    "plyer",
    "requests==2.32.3",
    "kivy-garden.mapview>=0.1.5",
]

[build-system]
requires = [
    "setuptools>=75.8.0",
    "cython>=0.29.36",
    "wheel"
]

[tool.uv.sources]
kivymd = { url = "https://github.com/kivymd/KivyMD/archive/master.zip" }
plyer = { url = "https://github.com/kivy/plyer/archive/master.zip" }
```

Figure 34: Mobile application dependencies with uv pyproject.toml

The tool used to build the mobile application was **Buildozer** (*Buildozer*, n.d.). To successfully compile the Python code and bundle it with all required dependencies into a file that can run on a mobile device, Buildozer requires a `.spec` configuration file that defines the build process, along with some system-level settings. The `.spec` file used for the build configuration is shown below:

```
[app]
title = DIT APP
package.name = dit_app
package.domain = org.dit_app
source.dir = .
source.include_exts = py,png,jpg,kv,atlas
source.include_patterns = *.py,*.kv,assets/*,images/*.png,fonts/*,data/*
source.exclude_patterns = Dockerfile,.dockerignore,.git/*
version = 0.1
requirements =
    python3,
```

```
    # KivyMD's direct dependencies (from uv tree)
    pillow==11.1.0,
    materialyoucolor==2.0.10,
    asynckivy==0.6.4,
    asyncgui==0.6.3,
    # Your app's direct dependencies
    https://github.com/kivy/plyer/archive/master.zip
    requests==2.32.3,
    # Kivy's dependencies (from uv tree)
    docutils==0.21.2,
    filetype==1.2.0,
    kivy-garden==0.1.5,
    pygments==2.19.1,
    # requests' sub-dependencies
    certifi==2025.1.31,
    charset-normalizer==3.4.1,
    idna==3.10,
    urllib3==2.3.0,
    # Build system
    setuptools==75.8.0,
    pip==25.0.1,
    cython==0.29.36,
    wheel,
    kivy==2.3.1,
    https://github.com/kivymd/KivyMD/archive/master.zip
orientation = portrait
android.kivy_version = master
author = © Copyright Info akotronis
fullscreen = 0
android.permissions = INTERNET, READ_EXTERNAL_STORAGE,
WRITE_EXTERNAL_STORAGE, ACCESS_FINE_LOCATION, ACCESS_COARSE_LOCATION
android.api = 34
android.minapi = 21
android.ndk = 25b
android.ndk_api = 21
android.entrypoint = org.kivy.android.PythonActivity
android.enable_androidx = True
android.archs = arm64-v8a
android.allow_backup = True
p4a.branch = develop
```

```
p4a.bootstrap = sdl2

[buildozer]
log_level = 2
warn_on_root = 1
```

Figure 35: Buildozer .spec file

To avoid manually managing the system dependencies required by Buildozer when building the app, we used the official Docker image provided in the kivy/buildozer repository (*Kivy/buildozer: Generic Python Packager for Android and iOS*, n.d.). The following commands were used to launch the build and run process using this image:

```
$ docker build -t img-dit-buildozer .
$ docker run --rm -it --entrypoint /bin/sh -v
"$HOME/.gradle":/home/user/.gradle -v
"$HOME/.buildozer":/home/user/.buildozer -v
"$(pwd)":/home/user/hostcwd img-dit-buildozer -c "buildozer -v
android debug"
```

These commands were used in conjunction with a custom *Dockerfile*:

```
FROM ghcr.io/kivy/buildozer

RUN pip install appdirs sh colorama jinja2 toml build
```

Figure 36: Dockerfile for the kivy/buildozer repository image

A key optimization here involves the use of volume mounts. Specifically, mounting `"$HOME/.gradle"` to `/home/user/.gradle` significantly reduces rebuild times—**from 20–50 minutes down to just 1–2 minutes** which is very crucial.

The complete codebase, along with additional implementation details, is available in the thesis GitHub repository (Kotronis, n.d.).

Chapter 6: Conclusion and Future Work

6.1: System Implementation Summary

This thesis presented the end-to-end design and development of a self-hosted geolocation sharing platform designed for closed user groups, focusing on privacy, modularity, and integration flexibility. At its core, the system relies on a mobile application built with Kivy/KivyMD, capable of collecting GPS data and transmitting it securely via WireGuard VPN tunnels to a Django-based backend with spatial data support. On the frontend, a Streamlit-based dashboard provides the ability to invite users to join specific groups, map visualization of live user locations, zone management with geofencing capabilities, and event-triggered notification mechanisms. The system also integrates with Home Assistant, enabling automated actions based on geolocation events.

Notably, the project succeeded in:

- Integrating a full VPN-based communication layer, with reverse proxies for external access (Ngrok and Localtonet).
- Achieving mobile-to-backend secure communication even across mobile networks, without requiring router port forwarding.
- Implementing geofencing logic and real-time zone-entry/exit detection using PostGIS spatial queries.
- Triggering physical smart home actions via Home Assistant automations (specifically controlling a TP-Link smart plug).
- Offering a clear and focused frontend for monitoring and control, including group/zone management, location sharing toggles, and live event visibility.
- Delivering a fully dockerized and reproducible deployment environment using Docker Compose and Vagrant for VM-based isolation.
- Ensuring the system runs **entirely self-hosted** without any reliance on cloud platforms, paid APIs, or proprietary services.
- Maintaining **complete extensibility**, as each component (mobile app, backend, frontend, automation logic) is independently replaceable, connected only through well-defined APIs and shared VPN context.

- Avoiding hidden or ongoing costs thanks to its open-source stack and ability to run entirely on local or virtualized infrastructure, without requiring hosting subscriptions or complex configuration.

Although designed as a proof-of-concept, the implementation demonstrates the feasibility of combining open-source technologies to build a location-aware IoT solution under user control. It covers end-to-end flows from user onboarding to smart device automation, and emphasizes modularity, transparency, and future extensibility.

6.2: Development Challenges

During the development of the system, several practical challenges arose, particularly related to compatibility between libraries, lack of up-to-date documentation, and deployment issues on Windows systems. The most significant difficulties are summarized in the following sections.

6.2.1: Kivy and KivyMD Compatibility and Versioning

Developing the mobile application using Kivy and KivyMD presented persistent compatibility issues. Frequent breaking changes between versions, especially in KivyMD, led to situations where updates caused code that used to work to stop working. Particularly, a lack of proper alignment between official sources such as PyPI, GitHub repositories and documentation was noted. Additionally, online resources often lag behind the latest versions, and best practices are inconsistently presented across community and official channels.

For instance, implementing the application build process with Buildozer proved complex. While **uv** was used on development for dependency management, the Buildozer build process relies on **pip** and handles dependencies through its own `.spec` file format, in a way that obscures how those contents are parsed and applied during the build. This mismatch led to inconsistencies and unpredictable behavior.

Additionally, due to the limited clarity in the official Buildozer documentation, initial build attempts failed. As a workaround, I resorted to using the Docker image provided in the kivy/buildozer repository (*Kivy/buildozer: Generic Python Packager for Android and iOS*, n.d.), which ultimately succeeded with some modifications. However, conflicting guidance from other

sources, such as the Android-for-Python Users repository (*Android-For-Python/Android-For-Python-Users: An Unofficial Buildozer Users' Guide*, 2023), cast doubt on whether this image-based approach is officially recommended or maintained, further complicating the process.

A specific case was encountered with the `plyer` library used for GPS access (*Plyer · PyPI*, n.d.). Although a bug in version 2.1.0 had been resolved in 2023 (*Example GPS Does Not Work on Newer Versions Android*, n.d.), the fix had not been published to PyPI at the time of development. As a result, considerable effort was required to locate and manually install a compatible version from the GitHub repository.

6.2.2: Home Assistant on Windows and Networking Issues

Initial attempts to run Home Assistant within a Docker container on Windows (via Docker Desktop and WSL) revealed serious limitations. Using `network_mode: host`, as commonly recommended, resulted in the Home Assistant interface being inaccessible via browser. Conversely, when reverting to port mapping, the browser could access the UI, but Home Assistant failed to discover smart devices on the network.

Investigations revealed that host networking requires Docker Desktop version ≥4.34 to function properly on Windows, as documented in the official Docker documentation (*Networking Using the Host Network*, n.d.). Even with this version, device discovery issues persisted, likely due to limitations in how Docker Desktop interacts with the Windows network stack.

Ultimately, to overcome these limitations, the entire microservices stack was migrated to a virtual machine using Vagrant and VirtualBox. This environment offered better network isolation and compatibility, enabling successful device discovery and reliable operation of Home Assistant with smart plugs like the TP-Link Tapo P110.

These difficulties highlight the broader challenge of building cross-platform IoT systems using open-source tools, where fragmented documentation, version inconsistencies, and platform-specific quirks can significantly increase development time and complexity.

6.3: Potential Improvements

6.3.1: Architecture and Implementation

Although the presented solution is largely self-contained, several aspects could be improved in terms of functionality and implementation, or extended with additional features.

Currently, the “real-time” aspect of the system relies on a **polling** technique. At predefined intervals, the frontend service sends requests to the backend to fetch the latest user location updates and geofencing notifications. However, this approach has some limitations.

The first issue is **inefficiency**: the frontend sends a request at every interval, regardless of whether new data is available. This shifts the responsibility for detecting changes from the backend (where the information originates) to the frontend (which needs the information), leading to unnecessary network overhead and affects scalability.

A more appropriate and efficient alternative is to **reverse these roles** using **WebSockets**. The Django framework supports this via the excellent **Django Channels** library (*Django Channels*, n.d.).

WebSockets establish a **bidirectional communication channel** between the frontend and backend, which remains open. This allows both sides to send and receive messages in real time. With this approach, the backend can **proactively** push updates to the frontend only when relevant events occur, such as when a new GPS location is received or when a user enters or exits a predefined zone.

6.3.2: New Features and Additions

A feature that is **partially implemented** is the support for **message exchange between group members** and the **corresponding** notifications in the dashboard. A dedicated **Chat** page is included in the Streamlit dashboard under the **Interactions** section on the left pane, and on the backend, a `Message` table is defined and associated with the `Notifications` table.

The most suitable approach for completing this feature would be to integrate **Django Channels** into the backend service to enable real-time communication.

Another potential improvement is the **full integration of dashboard functionality into the mobile application**. In the current implementation, the mobile app includes only the **Home**, **Login**, and **Registration** pages. Although users can access the dashboard from a mobile browser by visiting the backend's fixed container IP address, this approach has limitations. While Streamlit pages are natively responsive, the **map component has rendering issues**, and user interaction can be **unreliable in certain scenarios**.

The Kivy framework supports the implementation of interactive maps through the use of the `Mapview` widget provided by the **kivy-garden** package (*Kivy-Garden*, n.d.). In addition, online resources are available to guide developers through the implementation process (Sandberg, n.d.).

6.4: Closing Remarks

By bringing together a variety of open-source technologies into a well-integrated system, this thesis has demonstrated that self-hosted, privacy-preserving location-sharing platforms are viable, even for single developers. The resulting system fulfills its intended objectives and serves as a strong base for future enhancements in terms of scalability, user experience, and broader automation scenarios. The work undertaken lays the groundwork for exploring new directions while already delivering a solid and functional prototype.

References

Android-for-Python/Android-for-Python-Users: An unofficial Buildozer users' guide. (2023, November 14). GitHub. Retrieved June 12, 2025, from https://github.com/Android-for-Python/Android-for-Python-Users?tab=readme-ov-file#install

Brython. (n.d.). Brython. Retrieved June 10, 2025, from https://brython.info/

Buildozer. (n.d.). Welcome to Buildozer's documentation! — Buildozer 0.11 documentation. Retrieved June 13, 2025, from https://buildozer.readthedocs.io/en/latest/

Carrier-grade NAT. (n.d.). Wikipedia. Retrieved June 3, 2025, from https://en.wikipedia.org/wiki/Carrier-grade_NAT

Databases. (n.d.). Django documentation. Retrieved June 4, 2025, from https://docs.djangoproject.com/en/5.2/ref/databases/

Django Channels. (n.d.). Django Channels — Channels 4.2.0 documentation. Retrieved June 12, 2025, from https://channels.readthedocs.io/en/latest/

Django Integration With MongoDB Tutorial. (n.d.). MongoDB. Retrieved June 4, 2025, from https://www.mongodb.com/resources/products/compatibilities/mongodb-and-django

Django overview. (n.d.). Django. Retrieved June 4, 2025, from https://www.djangoproject.com/start/overview/

Django RepositoryStats. (n.d.). Retrieved 6 4, 2025, from https://repositorystats.com/django/django

djangorestframework-gis. (n.d.). Retrieved 6 4, 2025, from https://pypi.org/project/djangorestframework-gis/

draw.io. (n.d.). draw.io. Retrieved June 13, 2025, from https://www.drawio.com/

DrawSQL. (n.d.). DrawSQL. Retrieved June 5, 2025, from https://drawsql.app/diagrams

Example GPS does not work on newer versions android. (n.d.). Github. Retrieved 6 12, 2025, from https://github.com/kivy/plyer/issues/505

FastAPI RepositoryStats. (n.d.). Retrieved 6 4, 2025, from https://repositorystats.com/fastapi/fastapi

Flet - Python. (n.d.). Flet: Build multi-platform apps in Python powered by Flutter. Retrieved June 10, 2025, from https://flet.dev/

Frey, C. B., & Osborne, M. (n.d.). *Automation*. Wikipedia. Retrieved May 27, 2025, from https://en.wikipedia.org/wiki/Automation

Functional Requirement. (n.d.). Retrieved 5 29, 2025, from https://en.wikipedia.org/wiki/Functional_requirement

GeoDjango Tutorial. (n.d.). Django documentation. Retrieved June 4, 2025, from https://docs.djangoproject.com/en/5.2/ref/contrib/gis/tutorial/#introduction

Geofence. (n.d.). Wikipedia. Retrieved May 25, 2025, from https://en.wikipedia.org/wiki/Geofence

Geographical feature. (n.d.). Wikipedia. Retrieved May 25, 2025, from https://en.wikipedia.org/wiki/Geographical_feature

Golubic, K. (2023, June 5). *What is MIT License?* Memgraph. Retrieved June 3, 2025, from https://memgraph.com/blog/what-is-mit-license

Google Home. (n.d.). Manage Your Smart Home With Google Home. Retrieved June 12, 2025, from https://home.google.com/welcome/

Gradio - Python. (n.d.). Gradio App. Retrieved June 10, 2025, from https://www.gradio.app/

Harris, C. (n.d.). *Microservices vs. monolithic architecture*. Atlassian. Retrieved May 28, 2025, from https://www.atlassian.com/microservices/microservices-architecture/microservices-vs-monolith

Hejlsberg, A. (n.d.). Object–relational mapping - Wikipedia. Retrieved June 4, 2025, from https://en.wikipedia.org/wiki/Object%E2%80%93relational_mapping

Home Assistant Installation. (n.d.). Home Assistant. Retrieved June 13, 2025, from https://www.home-assistant.io/installation/linux#docker-compose

Home automation. (n.d.). Wikipedia. Retrieved May 27, 2025, from https://en.wikipedia.org/wiki/Home_automation

Install Docker Engine on Ubuntu. (n.d.). Docker Docs. Retrieved June 13, 2025, from https://docs.docker.com/engine/install/ubuntu/#uninstall-old-versions

Install the Docker Compose plugin. (n.d.). Docker Docs. Retrieved June 13, 2025, from https://docs.docker.com/compose/install/linux/#install-using-the-repository

Janetakis, N. (2024, April 2). *Connect to a Service Running on Your Docker Host from a Container — Nick Janetakis*. Nick Janetakis. Retrieved June 13, 2025, from https://nickjanetakis.com/blog/connect-to-a-service-running-on-your-docker-host-from-a-container

kivy/buildozer: Generic Python packager for Android and iOS. (n.d.). GitHub. Retrieved June 12, 2025, from https://github.com/kivy/buildozer

Kivy Docs - Philosophy. (n.d.). Retrieved 6 3, 2025, from https://kivy.org/doc/stable/philosophy.html#

kivy-garden. (n.d.). kivy-garden - mapview. Retrieved 6 12, 2025, from https://github.com/kivy-garden/mapview

Kivy · PyPI. (n.d.). PyPI. Retrieved June 3, 2025, from https://pypi.org/project/Kivy/#history

Kotronis, A. (n.d.). *akotronis/DIT-thesis*. akotronis/DIT-thesis. Retrieved 6 13, 2025, from https://github.com/akotronis/DIT-thesis

linuxserver/wireguard. (n.d.). Github. Retrieved 5 26, 2025, from https://github.com/linuxserver/docker-wireguard?tab=readme-ov-file#usage

localtonet - Docs. (n.d.). Retrieved 6 3, 2025, from https://localtonet.com/documents/using-localtonet-with-cgnat

Location-based service. (n.d.). Wikipedia. Retrieved May 25, 2025, from https://en.wikipedia.org/wiki/Location-based_service

Mercury - Python. (n.d.). Retrieved 6 10, 2025, from https://runmercury.com/

Networking using the host network. (n.d.). Docker Docs. Retrieved June 12, 2025, from https://docs.docker.com/engine/network/tutorials/host/#prerequisites

ngoduykhanh/wireguard-ui: Wireguard web interface. (n.d.). GitHub. Retrieved May 30, 2025, from https://github.com/ngoduykhanh/wireguard-ui?utm_source=chatgpt.com

NiceGUI - Python. (n.d.). NiceGUI. Retrieved June 10, 2025, from https://nicegui.io/

Node-RED. (n.d.). Node-RED: Low-code programming for event-driven applications. Retrieved June 12, 2025, from https://nodered.org/

Non-functional requirement. (n.d.). Wikipedia. Retrieved May 29, 2025, from https://en.wikipedia.org/wiki/Non-functional_requirement

openHAB. (n.d.). openHAB. Retrieved June 12, 2025, from https://www.openhab.org/

OpenVPN. (n.d.). OpenVPN: Business VPN For Secure Networking. Retrieved May 26, 2025, from https://openvpn.net/

Panel - Python. (n.d.). Overview — Panel v1.7.1. Retrieved June 10, 2025, from https://panel.holoviz.org/

plyer · PyPI. (n.d.). PyPI. Retrieved June 12, 2025, from https://pypi.org/project/plyer/

PostgreSQL - About. (n.d.). PostgreSQL. Retrieved June 5, 2025, from https://www.postgresql.org/about/

Protasiewicz, J. (2025, March 26). *Top 10 Django Apps Examples*. Netguru. Retrieved June 4, 2025, from https://www.netguru.com/blog/django-apps-examples

PyGWalker: Turn your data into visual analytic app. (n.d.). Kanaries. Retrieved June 10, 2025, from https://kanaries.net/pygwalker

Pyodide. (n.d.). Pyodide — Version 0.27.7. Retrieved June 10, 2025, from https://pyodide.org/en/stable/

PyScript. (n.d.). PyScript is an open source platform for Python in the browser. Retrieved June 10, 2025, from https://pyscript.net/

Python PEP 518. (n.d.). PEPS - Python. Retrieved 6 13, 2025, from https://peps.python.org/pep-0518/#file-format

Rodríguez, A., & Bulgakov, A. (n.d.). Welcome to KivyMD's documentation! — KivyMD 2.0.1.dev0 documentation. Retrieved June 3, 2025, from https://kivymd.readthedocs.io/en/latest/

Sandberg, E. (n.d.). *Simple & Elegant Map in Kivy - MapView Tutorial*. Youtube. Retrieved 6 12, 2025, from https://www.youtube.com/watch?v=P940dd1VxsU

Shiny for Python. (n.d.). Shiny for Python. Retrieved June 10, 2025, from https://shiny.posit.co/py/

SoftEther. (n.d.). SoftEther VPN Project - SoftEther VPN Project. Retrieved May 26, 2025, from https://www.softether.org/

streamlit-folium. (n.d.). streamlit-folium documentation. Retrieved June 10, 2025, from https://folium.streamlit.app/

StrongSwan. (n.d.). strongSwan - IPsec VPN for Linux, Android, FreeBSD, macOS, Windows. Retrieved May 26, 2025, from https://strongswan.org/

Taipy — Build Python Data & BI web applications. (n.d.). Taipy — Build Python Data & BI web applications. Retrieved June 29, 2025, from https://taipy.io/

Technology | 2024 Stack Overflow Developer Survey. (n.d.). Stack Overflow Annual Developer Survey. Retrieved June 5, 2025, from https://survey.stackoverflow.co/2024/technology#most-popular-technologies-database

Top 8 PostgreSQL Extensions. (n.d.). Retrieved 6 5, 2025, from https://www.timescale.com/blog/top-8-postgresql-extensions#top-8-postgresql-extensions-you-should-know-about-object-object

uv. (n.d.). Astral Docs. Retrieved June 12, 2025, from https://docs.astral.sh/uv/

Virtual private network. (n.d.). Wikipedia. Retrieved May 25, 2025, from https://en.wikipedia.org/wiki/Virtual_private_network

What is a Geofence? How does Geofencing work? (n.d.). Ryte. Retrieved May 25, 2025, from https://en.ryte.com/wiki/Geofencing/

What is a Virtual Machine? (n.d.). VMware. Retrieved June 11, 2025, from https://www.vmware.com/topics/virtual-machine

What is a VPN? Why Should I Use a VPN? (n.d.). Microsoft Azure. Retrieved May 25, 2025, from https://azure.microsoft.com/en-us/resources/cloud-computing-dictionary/what-is-vpn

What Is Containerization? (n.d.). IBM. Retrieved May 28, 2025, from https://www.ibm.com/think/topics/containerization

What is Containerization? - Containerization Explained. (n.d.). AWS. Retrieved May 28, 2025, from https://aws.amazon.com/what-is/containerization/

What is Geospatial Data? (n.d.). IBM. Retrieved June 4, 2025, from https://www.ibm.com/think/topics/geospatial-data